

Table of Contents

SOFTWARE DESIGN & ARCHITECTURE.....	6
EXECUTIVE SUMMARY	6
FOUNDATIONS OF SOFTWARE ENGINEERING	6
<i>Defining Key Terms</i>	6
<i>The Software Development Lifecycle (SDLC)</i>	6
ENGINEERING PROBLEM-SOLVING	7
<i>The Three States of Problem-Solving</i>	7
<i>Overcoming Functional Fixedness</i>	7
SOFTWARE ENGINEERING DESIGN: PROCESS & PRODUCT	7
<i>Benefits of Studying Design</i>	7
MAJOR SOFTWARE DESIGN CHALLENGES	8
THE SOFTWARE DESIGN PROCESS	8
<i>Architecture vs. Detailed Design</i>	8
<i>Specialized Design Activities</i>	8
CORE DESIGN PRINCIPLES (FUNDAMENTALS)	9
SPECIALIZED ROLES IN DESIGN	9
STUDY GUIDE.....	9
<i>Part 1: Short-Answer Quiz</i>	9
<i>Part 2: Quiz Answer Key</i>	10
<i>Part 3: Essay Questions</i>	11
GLOSSARY OF KEY TERMS.....	13
FUNDAMENTALS, PROCESS, & REQUIREMENTS ENGINEERING	15
EXECUTIVE SUMMARY	15
1. FUNDAMENTALS OF SOFTWARE ARCHITECTURE	15
<i>Definition & Dimensions</i>	15
<i>Process vs. Product</i>	15
<i>Scope: What Architecture is Not</i>	15
2. COMPARISON: ARCHITECTURE VS. DETAILED DESIGN	15
3. THE SOFTWARE ARCHITECTURE PROCESS.....	16
<i>Problem-Solving Framework</i>	16
4. REQUIREMENTS ENGINEERING FOR ARCHITECTS	16
<i>The Four Activities of Requirements Engineering</i>	16
<i>Requirement Classification</i>	17
<i>Desired Characteristics of Requirements</i>	17
5. QUALITY ATTRIBUTES & TACTICS	17
<i>Quality Tags</i>	17
<i>Design Tactics</i>	17
6. KEY ARCHITECTURAL TASKS & CONCEPTS.....	18
<i>Architectural Views</i>	18
<i>Styles & Patterns</i>	18
<i>Design Synchronicity</i>	18

STUDY GUIDE.....	18
<i>Part I: Knowledge Review Quiz</i>	18
<i>Part II: Answer Key</i>	20
<i>Part III: Essay Format Questions</i>	20
GLOSSARY OF KEY TERMS	22
SOFTWARE ARCHITECTURE & QUALITY ATTRIBUTES	24
EXECUTIVE SUMMARY	24
1. FUNDAMENTAL CONCEPTS OF ARCHITECTURE & REQUIREMENTS	24
<i>The Role of Architecture</i>	24
2. CHARACTERIZING QUALITY VIA SCENARIOS	24
<i>The Six Parts of a Quality Attribute Scenario</i>	25
3. SYSTEMATIC QUALITY DESIGN DECISIONS.....	25
4. ANALYSIS OF MAJOR QUALITY ATTRIBUTES	25
<i>Availability</i>	25
<i>Modifiability</i>	26
<i>Performance</i>	26
<i>Security</i>	26
<i>Testability</i>	27
<i>Usability</i>	27
5. ARCHITECTURAL TACTICS SUMMARY TABLE	27
STUDY GUIDE.....	27
<i>Quality Attributes Quiz</i>	27
<i>Answer Key</i>	29
<i>Essay Questions</i>	29
GLOSSARY OF KEY TERMS	31
SOFTWARE ARCHITECTURE STYLES & PATTERNS.....	32
EXECUTIVE SUMMARY	32
DEFINING ARCHITECTURAL STYLES & PATTERNS	32
<i>The Benefits of Pattern-Based Design</i>	32
THE EVOLUTION OF ARCHITECTURAL DOCUMENTATION	33
CLASSIFICATION OF ARCHITECTURAL SYSTEMS	33
DETAILED ANALYSIS: DATA-CENTERED SYSTEMS	33
<i>The Blackboard Architectural Pattern</i>	33
DETAILED ANALYSIS: DATA FLOW SYSTEMS.....	34
<i>Pipes-&-Filters Pattern</i>	34
DISTRIBUTED & INTERACTIVE SYSTEMS	34
<i>Interactive (Local) Systems</i>	35
<i>Distributed (Networked) Systems</i>	35
HIERARCHICAL SYSTEMS.....	35
SUMMARY CHECKLIST FOR PATTERN SELECTION	35
STUDY GUIDE.....	35
<i>Part I: Short-Answer Quiz</i>	35
<i>Part II: Quiz Answer Key</i>	37
<i>Part III: Essay Format Questions</i>	37

GLOSSARY OF KEY TERMS	39
PRINCIPLES & PRACTICES OF SOFTWARE DETAILED DESIGN	40
EXECUTIVE SUMMARY	40
OVERVIEW OF DETAILED DESIGN	40
<i>The Design Hierarchy</i>	40
<i>The Designer's Mental Model</i>	40
KEY TASKS IN DETAILED DESIGN	40
COMPONENT DESIGN IN OBJECT-ORIENTED (OO) SYSTEMS	41
<i>Structural vs. Behavioral Modeling</i>	41
<i>Abstract Classes & Interfaces</i>	41
THE SOLID PRINCIPLES OF INTERNAL DESIGN	41
1. <i>Single Responsibility Principle (SRP)</i>	42
2. <i>Open-Closed Principle (OCP)</i>	42
3. <i>Liskov Substitution Principle (LSP)</i>	42
EVALUATION & DOCUMENTATION STANDARDS	42
<i>Technical Reviews</i>	42
<i>The Software Design Document (SDD)</i>	43
MANAGEMENT & IMPLEMENTATION	43
<i>Fundamental Rules of Design</i>	43
STUDY GUIDE	43
<i>Part 1: Comprehensive Review</i>	43
<i>Key Tasks in Detailed Design</i>	44
<i>Documentation & Evaluation</i>	44
<i>Implementation Management & Synchronicity</i>	44
<i>Component-Level Design</i>	45
<i>SOLID Principles (Initial Concepts)</i>	45
<i>Part 2: Short-Answer Quiz</i>	45
<i>Part 3: Answer Key</i>	46
<i>Part 4: Essay Format Questions</i>	47
GLOSSARY OF KEY TERMS	48
DETAILED ANALYSIS OF CREATIONAL DESIGN PATTERNS & THE ABSTRACT FACTORY	49
EXECUTIVE SUMMARY	49
FOUNDATIONS OF DESIGN PATTERNS	49
<i>Classification Criteria</i>	49
<i>Documenting Patterns</i>	49
THE ABSTRACT FACTORY PATTERN	50
<i>Core Intent & Structure</i>	50
<i>Key Benefits</i>	50
IMPLEMENTATION METHODOLOGY	50
ILLUSTRATIVE CASE STUDIES	51
1. <i>User Interface (UI) Themes</i>	51
2. <i>Computer Store System</i>	51
3. <i>Automotive Branch Management (RiyadhHQ)</i>	51
EVALUATIVE ANALYSIS	52

STUDY GUIDE.....	52
<i>Part 1: Short-Answer Quiz</i>	52
<i>Part 2: Answer Key</i>	53
<i>Part 3: Essay Questions</i>	53
GLOSSARY OF KEY TERMS	55
STRUCTURAL DESIGN PATTERNS: ADAPTER, COMPOSITE, & FACADE	56
EXECUTIVE SUMMARY	56
THE ADAPTER DESIGN PATTERN	56
<i>Intent & Mechanism</i>	56
<i>Structural Variations</i>	56
<i>Case Study: Bird to ToyDuck Adaptation</i>	57
<i>Evaluation</i>	57
THE COMPOSITE DESIGN PATTERN	57
<i>Core Components</i>	57
<i>Implementation Requirements</i>	57
<i>Real-Life Applications</i>	58
<i>Benefits</i>	58
THE FACADE DESIGN PATTERN	58
<i>Intent & Implementation</i>	58
<i>Case Study: Mobile Shop Subsystem</i>	58
<i>Evaluation of Benefits</i>	58
CONCLUSION.....	59
STUDY GUIDE.....	59
<i>Short-Answer Quiz</i>	59
<i>Answer Key</i>	60
<i>Essay Questions</i>	60
GLOSSARY OF KEY TERMS	62
SOFTWARE ARCHITECTURE EVALUATION	63
EXECUTIVE SUMMARY	63
FUNDAMENTALS OF ARCHITECTURE EVALUATION.....	63
<i>Benefits of Systematic Evaluation</i>	63
<i>Major Evaluation Methods</i>	63
THE ARCHITECTURE TRADEOFF ANALYSIS METHOD (ATAM)	64
<i>Quality Attribute Perspectives</i>	64
<i>Participant Groups & Roles</i>	64
THE ATAM PROCESS: STEPS & PHASES	64
<i>Execution Phases</i>	64
<i>The 9-Step Methodology</i>	65
ANALYSIS ARTIFACTS: RISKS, SENSITIVITIES, & TRADEOFFS.....	65
LIGHTWEIGHT ATAM.....	65
<i>Key Differences:</i>	65
REQUIRED EVALUATION OUTPUTS	66
STUDY GUIDE.....	66
<i>Part 1: Short-Answer Quiz</i>	66

<i>Part 2: Answer Key</i>	67
<i>Part 3: Essay Questions</i>	68
GLOSSARY OF KEY TERMS	70
SOFTWARE ARCHITECTURE DOCUMENTATION	71
EXECUTIVE SUMMARY	71
THE STRATEGIC IMPORTANCE OF DOCUMENTATION	71
<i>Primary Utility:</i>	71
FRAMEWORKS FOR ARCHITECTURAL VIEWS	71
<i>Common Architectural View Approaches</i>	72
<i>The 4+1 View Model Detail</i>	72
THE DOCUMENTATION PROCESS	72
<i>Rational Unified Process (RUP) Template Structure</i>	72
DETAILED COMPONENTS OF A SINGLE VIEW	73
DOCUMENTATION BEYOND THE VIEW	73
DOCUMENTING QUALITY ATTRIBUTES	73
STUDY GUIDE	74
<i>Part 1: Short-Answer Quiz</i>	74
<i>Part 2: Quiz Answer Key</i>	74
<i>Part 3: Essay Questions</i>	75
GLOSSARY OF KEY TERMS	77

Software Design & Architecture

Executive Summary

Software engineering design is a systematic, disciplined, & quantifiable approach to creating high-quality software systems. Functioning as the foundation of the software development lifecycle (SDLC), effective design mitigates the risks of bugs, crashes, & unmaintainable code. The discipline is characterized by a dual nature, serving as both a **process** of organized activities & a **product** consisting of technical artifacts like UML diagrams.

Key takeaways from this analysis include:

- **The Foundation Analogy:** Building software is compared to constructing a house; a weak architectural foundation leads to eventual failure regardless of aesthetic appeal.
- **Problem-Solving Frameworks:** Successful design requires a holistic approach to problem-solving, moving through initial (interpretation), operational (brainstorming), & goal (implementation) states while overcoming “functional fixedness.”
- **Architecture vs. Detailed Design:** Architecture focuses on the “macro” view & non-functional quality attributes (security, scalability), while detailed design focuses on “micro” internal logic & functional requirements.
- **Core Principles:** The “modularization engine” is driven by abstraction (hiding unnecessary details) & encapsulation (restricting access), aimed at achieving low coupling & high cohesion.
- **Persistent Challenges:** Designers must navigate requirements volatility, rapid technological evolution, conflicting stakeholder influences, & ethical responsibilities.

Foundations of Software Engineering

Software engineering emerged in 1968 to address the increasing complexity & failure rates of software projects in the 1960s. It adapts traditional engineering methods to the digital realm.

Defining Key Terms

- **Software:** A collection of instructions enabling users to interact with hardware to perform specific tasks (e.g., operating systems, web browsers).
- **Engineering:** A systematic, organized, & logical process used to generate & evaluate designs that meet client objectives & user needs within specified constraints.
- **Software Engineering (IEEE Definition):** The application of a systematic, disciplined, & quantifiable approach to the development, operation, & maintenance of software.

The Software Development Lifecycle (SDLC)

The process is categorized into five primary phases:

1. **Requirements:** Analyzing, specifying, & validating stakeholder needs.

2. **Design:** Using specifications to create software architecture & detailed models.
3. **Implementation:** Translating designs into executable code using programming languages.
4. **Testing:** Verifying that the software meets requirements & behaves correctly.
5. **Maintenance:** Modifying software post-delivery to correct faults or adapt to new environments.

Engineering Problem-Solving

Design is fundamentally a problem-solving activity. Engineers must identify, evaluate, & propose solutions to complex issues through a structured framework.

The Three States of Problem-Solving

State	Activities
Initial State	Problem formulation & interpretation. Understanding the problem is often a challenge in itself.
Operational State	Brainstorming, thinking about viable solutions, & synthesizing possibilities.
Goal State	Identifying, evaluating, & validating the final solution.

Overcoming Functional Fixedness

A significant barrier in the operational state is **functional fixedness**, a mental block where individuals limit an object's use to its traditional function. Designers are encouraged to “think outside the box” & consider problems from unusual angles to find innovative solutions.

Software Engineering Design: Process & Product

In the industry, “design” is used interchangeably to describe both the activities performed & the resulting artifacts.

- **Process Perspective:** Focuses on project management aspects, including planning, organizing, & assigning tasks to ensure resources are aligned before construction.
- **Product Perspective:** Focuses on technical aspects, resulting in artifacts such as class diagrams, sequence diagrams, & design documents.

Benefits of Studying Design

- **Management Benefits:** Minimizes the effects of requirements volatility, optimizes human resource allocation, & prevents “single-point-of-failure” where one individual owns the entire system knowledge.
- **Technical Benefits:** Maps requirements to conceptual models, identifies main components & interfaces, allows for the evaluation of quality attributes (usability, efficiency), & enables the reuse of solutions in future projects.

Major Software Design Challenges

Modern software engineers face five significant challenges that complicate the development of high-quality systems:

1. **Requirements Volatility:** Changes in requirements late in the lifecycle are costly, requiring extensive rework of design, verification, & deployment plans.
2. **Inconsistent Development Processes:** Teams within the same organization may follow different methodologies (e.g., one using Agile, another Waterfall), leading to integration delays & mismatched systems.
3. **Fast-Changing Technology:** The rapid evolution of technologies like AI & Machine Learning requires designers to learn & implement new concepts quickly while ensuring interoperability with legacy systems.
4. **Managing Design Influences:** Stakeholders often have conflicting priorities (e.g., executives wanting profit-driven features vs. security teams demanding protection), requiring difficult design trade-offs.
5. **Ethics & Professionalism:** Designers must consider how their decisions affect society. Guidelines like the **ACM Code of Ethics** emphasize privacy, honesty, & prioritizing the public good.

The Software Design Process

The design process is divided into major & supportive activities that bridge the gap between requirements & construction.

Architecture vs. Detailed Design

Aspect	Software Architecture	Detailed Design
Focus	Macro-design; major components & quality.	Micro-design; internal logic & behavior.
Viewpoint	Black-box: External properties & interactions.	White-box: Internal structure & algorithms.
Requirements	Primarily Non-functional (Security, Scalability).	Primarily Functional (Logic, Data handling).
Timing	Foundation for subsequent work.	Starts after architecture is approved.

Specialized Design Activities

- **Component Design:** Creating the internal structure (e.g., UML class diagrams) of components identified during the architecture phase.
- **Interface Design:** Specifying how components interact internally or externally with third-party systems (e.g., payment gateways).
- **Construction Design:** The lowest level of design, focusing on algorithm analysis & implementation details.

- **Human-Computer Interface (HCI) Design:** Optimizing the interaction between users & software to ensure usability & efficiency.

Core Design Principles (Fundamentals)

These principles guide the “modularization engine” to create manageable, high-quality systems.

1. **Modularization:** The process of decomposing a system into small, fine-grained components with focused responsibilities. This makes projects easier to build, test, & maintain.
2. **Abstraction:** Hiding unnecessary details to focus on essential characteristics.
 - *Procedural Abstraction:* Simplifying complex behavioral operations (e.g., a “SEND” comm& hiding TCP/IP connection steps).
 - *Data Abstraction:* Simplifying the structural composition of data objects.
3. **Encapsulation:** Protecting hidden parts of an abstraction. It ensures that entities only expose essential information through controlled interfaces (e.g., private variables accessed via public methods).
4. **Coupling:** The degree of interdependence between modules. Designers aim for **Low Coupling** to minimize dependencies.
 - *Content Coupling (Avoid):* One module modifies the internal data of another.
 - *Common Coupling (Avoid):* Multiple modules rely on global variables.
5. **Sufficiency:** Ensuring a unit provides *only* the services necessary for its intent (no more).
6. **Completeness:** Ensuring a unit provides *all* the services required to achieve its intent (no less).

Specialized Roles in Design

Effective software design involves various professional roles with distinct focuses:

- **Systems Engineer:** Designs the overall process for both hardware & software.
- **Software Architect:** Focuses on high-level structure & quality attributes (black-box approach).
- **Component Designer:** Designs & implements the internal logic of specific modules (white-box approach).
- **User Interface (UI) Designer:** Specializes in system usability & the user experience.

Study Guide

This study guide provides a comprehensive overview of the principles, processes, & challenges inherent in software design & architecture. It is designed to facilitate a deep understanding of how systematic engineering principles are applied to create reliable, maintainable software systems.

Part 1: Short-Answer Quiz

Instructions: Answer the following questions in 2–3 sentences based on the provided materials.

1. According to the IEEE, how is software engineering formally defined?
2. What is the significance of the “building a house” analogy in the context of software development?
3. In the software industry, what is the difference between the “process” & “product” perspectives of design?
4. How does “functional fixedness” impact the engineering problem-solving process?
5. Why is “Requirements Volatility” considered a major challenge in software design?
6. Distinguish between the roles of a Software Architect & a Component Designer.
7. What are the three states of the engineering problem-solving process?
8. Explain the relationship between the principles of Abstraction & Encapsulation.
9. What is the primary difference between Software Architecture & Detailed Design regarding their focus on requirements?
10. Define “Common Coupling” & explain why it can be problematic for a system.

Part 2: Quiz Answer Key

1. **IEEE Definition:** Software engineering is the application of a systematic, disciplined, & quantifiable approach to the development, operation, & maintenance of software. Essentially, it represents the application of traditional engineering principles to the creation of software.
2. **House Analogy:** This analogy emphasizes that the foundation of software consists of its architecture & design principles. Just as a weak foundation causes a house to crack regardless of its beauty, poor software architecture leads to bugs, crashes, & unmaintainable code over time.
3. **Process vs. Product:** From a process perspective, design refers to the activities & tasks required to model a system’s structure & behavior before construction, often linked to project management. From a product perspective, it refers to the resulting technical artifacts, such as class diagrams or design documents.
4. **Functional Fixedness:** Functional fixedness is a cognitive bias that limits a person to using an object only in the way it is traditionally used, which can hinder the ability to find creative solutions. Overcoming this is critical during the operational state of problem-solving to ensure designers can “think outside the box” & evaluate problems from unusual angles.
5. **Requirements Volatility:** This refers to the growth or change of requirements during the development lifecycle, which is highly challenging because changes in later stages are extremely costly. Such shifts often require the rework of the design, verification processes, & deployment plans.
6. **Architect vs. Component Designer:** A Software Architect uses a “black-box” approach to design high-level structures, focusing on external properties & quality attributes. A Component Designer

focuses on the “white-box” internal structure & behavior of specific modules, typically requiring strong programming skills for implementation.

7. **Problem-Solving States:** The process begins with the **Initial State**, where problems are formulated & interpreted. It moves to the **Operational State**, where viable solutions are brainstormed & evaluated, & concludes with the **Goal State**, where an appropriate solution is validated & implemented.
8. **Abstraction & Encapsulation:** These principles work h& in h&: abstraction focuses on identifying the essential characteristics of an entity while hiding unnecessary details. Encapsulation then enforces these abstractions by protecting the hidden details & only exposing essential information through controlled interfaces.
9. **Focus on Requirements:** Software Architecture focuses primarily on non-functional or quality requirements, such as security, performance, & scalability. In contrast, Detailed Design focuses on functional requirements, specifying the exact features, logic, & internal workflows of the system components.
10. **Common Coupling:** Common coupling occurs when multiple modules depend on a shared global data area or global variables. This is problematic because any change to the global area can cause unexpected side effects across all dependent modules, making the system difficult to maintain.

Part 3: Essay Questions

Instructions: Use the source context to develop detailed responses to the following prompts.

1. **The Evolution of Software Engineering:** Discuss the historical context of the 1960s that led to the formal introduction of software engineering in 1968. Explain why a systematic, disciplined, & quantifiable approach became necessary.
2. **The Holistic Problem-Solving Framework:** Detail the tasks involved in the holistic approach to engineering problem-solving. How do these tasks map to the Initial, Operational, & Goal states?
3. **Design as a Management Tool:** Analyze how software design benefits project management. Specifically, address how it minimizes the effects of requirements volatility & prevents the “one guy” syndrome where a single individual owns the knowledge of the entire system.
4. **Major Design Challenges in Modern Software:** Compare & contrast the challenges of “Inconsistent Development Processes” & “Fast, Ever-Changing Technology.” How does each affect the ability of a team to deliver a high-quality product on time?

5. **Ethics & Professionalism in Design:** Using the ACM Code of Ethics as a reference, discuss the ethical responsibilities of a software designer. How do these principles influence interactions with stakeholders & the development team under pressure?

Glossary of Key Terms

Term	Definition
Abstraction	The principle of focusing on the essential characteristics of an entity while deferring or hiding unnecessary details.
ACM Code of Ethics	A set of general ethical principles & professional responsibilities, such as respecting privacy & prioritizing the public good, that guide software designers.
Black-Box View	A design perspective that focuses on what a component does & how it interacts with others, rather than its internal workings.
Cohesion	A measure of how strongly related & focused the responsibilities of a single software module are.
Component Design	An activity within detailed design that specifies the internal structure & behavior (e.g., class diagrams) of individual modules.
Construction Design	The lowest level of detailed design, dealing with the analysis & design of specific algorithms & logic for function implementations.
Coupling	The degree of interdependence between software modules; designers generally aim for “low” or “loose” coupling.
Encapsulation	The principle of hiding the internal implementation of an entity & restricting access to only essential information through a well-defined interface.
Engineering	A systematic, organized, & structured process used to generate & evaluate designs that meet client objectives while satisfying constraints.
Functional Abstraction	Also known as procedural abstraction; it simplifies reasoning by using a single term (like “SEND”) to represent a complex sequence of behavioral steps.
HCI Design	Human-Computer Interface design; the activity of optimizing the interaction between users & software to ensure usability & efficiency.
IEEE	The Institute of Electrical & Electronics Engineers, a professional organization that sets standards for software engineering.
Modularization	The process of decomposing a software system into smaller, separate components (fine-grained modules) to make the system more manageable.
Software	A collection of instructions that enable a user to interact with computer hardware to perform specific tasks.
Software Architecture	The high-level “macro-design” of a system that provides the major structural components, interfaces, & foundation for all subsequent work.
Structured Design	A design strategy that breaks a system into independent, single-purpose modules using a top-down approach focused on functions.

Sufficiency	A design principle that measures whether a design unit provides only the services necessary to achieve its intent, & nothing more.
Systems Engineer	A role responsible for designing the overall development process for a system as a whole, including both hardware & software.
White-Box View	A design perspective that focuses on the internal structure, algorithms, & data handling of a component.

Fundamentals, Process, & Requirements Engineering

Executive Summary

Software architecture serves as the high-level structural blueprint of a system, focusing on major components, their interactions, & non-functional requirements. It is a foundational, non-optional activity that translates requirements into a graphical, design-oriented domain. Unlike detailed design, which handles internal logic & class-level implementation, architecture provides the “big picture” necessary to meet quality goals such as performance, security, & scalability.

The success of an architectural effort depends heavily on **Requirements Engineering**, which involves eliciting & analyzing stakeholder concerns to drive design decisions. A critical aspect of this process is the identification of **quality attributes** & the implementation of specific **tactics** to achieve them. Furthermore, maintaining **design synchronicity**—the alignment between the architectural vision & the final code—is essential to prevent system failure & excessive costs during later development stages.

1. Fundamentals of Software Architecture

Definition & Dimensions

Software architecture is defined as the high-level structure of a software system, the discipline of creating those structures, & their documentation. It captures three primary dimensions:

- **Structure:** The “parts” or components of the system.
- **Communication (Behavior):** How the parts fit & talk together.
- **Nonfunctional Requirements:** The quality attributes the system must exhibit.

Process vs. Product

Software architecture is categorized as both an activity & a result:

- **The Process:** The foundational design activity that evaluates & translates functional & non-functional requirements into design elements.
- **The Product:** The resulting high-level diagrams, component models, & blueprints that support detailed design & construction.

Scope: What Architecture is Not

Architecture focuses on aspects of a system that are difficult to change once built. It is specifically **not** concerned with:

- Development/Coding
- Internal data
- Algorithms.
- structures.

2. Comparison: Architecture vs. Detailed Design

Architecture & design differ in their level of abstraction & interaction focus.

Feature	Software Architecture (Big Picture)	Software Design (Detailed Picture)
Focus	Major components (Frontend, Backend, Database).	Internal details (Classes, methods, logic).
Interaction	Component-to-component communication.	Object-to-object / class-to-class interaction.
Example	Deciding the system needs a Payment Service.	Designing the addItem() method in a ShoppingCart class.
Products	High-level blueprints & subsystem diagrams.	Class diagrams, sequence diagrams, & flowcharts.

3. The Software Architecture Process

The creation of software architecture is a large-scale, iterative process involving five core tasks:

1. **Understand & Evaluate Requirements:** Engaging in requirements engineering to define the system's needs.
2. **Design the Architecture:** Identifying major components, views, styles, & patterns.
3. **Evaluate the Architecture:** A lengthy, iterative process to catch defects early, when they are cheaper to correct.
4. **Document the Architecture:** Capturing the design in a format reviewable by stakeholders.
5. **Monitor & Control Implementation:** Ensuring the system's construction aligns with the architectural vision.

Problem-Solving Framework

Architectural design is driven by problem-solving under constraints. The architect must interpret requirements & scenarios while evaluating constraints (schedule, cost, & quality) through collaborative brainstorming & design reviews.

4. Requirements Engineering for Architects

Architects must be proficient in requirements engineering, as architectural design begins with understanding the context & boundaries of the system.

The Four Activities of Requirements Engineering

- **Elicitation:** Identifying stakeholders & uncovering their needs through interviews, meetings, observations, & scenarios.
- **Analysis:** Addressing raw requirements that are contradictory, incomplete, or vague. This includes classification, prioritization, & negotiation.
- **Specification:** Formally capturing the results in an appropriate format (e.g., Software Requirements Specification).
- **Validation:** Reviewing requirements with stakeholders to ensure accuracy & feasibility.

Requirement Classification

Requirements are categorized to understand their nature:

- **Functional vs. Non-functional:** What the system does vs. how it behaves (quality goals).
- **Product vs. Process:** What the software must do vs. the methodology used to develop it (e.g., Agile).
- **Imposed vs. Derived:** Stakeholder-given requirements vs. those identified by the team to meet higher goals.

Desired Characteristics of Requirements

To be successful, requirements must be:

- **Specific:** Clear, concise, & exclusive (not open to interpretation).
- **Consistent:** They must not conflict with or prevent the implementation of other requirements.
- **Correct:** Accurately describing a desired system function.
- **Attainable:** Realistic within the constraints of time, technology, & budget.
- **Complete:** Thoroughly describing functions individually & as a collective set.
- **Verifiable:** Possible to measure or test to confirm the requirement is met.

5. Quality Attributes & Tactics

Architecture focuses on the qualities (performance, security, etc.) provided by major components.

Quality Tags

It is insufficient to simply identify a component; architects must “tag” components with quality properties.

- **Example:** A component providing a search algorithm should be tagged with its performance rating (e.g., $O(n^2)$) so clients understand its limitations.

Design Tactics

A **tactic** is a specific design decision made to achieve or control a quality attribute response.

Quality Attribute	Description	Example Tactics
Security	Protecting & defending information.	User authentication, limiting access on a “need-to-know” basis.
Performance	Capacity to work under time/resource constraints.	Caching, introducing concurrency, reducing overhead.
Availability	System uptime.	Redundancy, task monitors, watchdog timers.
Modifiability	Ease of changing the system for future needs.	Modularization, encapsulation, reducing coupling.

Testability	Ease of verifying & validating functions.	Event logging (allowing testers to trace operations).
Usability	Complexity involved in learning/using the system.	Supporting “undo” operations, providing a “cancel” option.

6. Key Architectural Tasks & Concepts

Architectural Views

No single diagram can describe an entire architecture. Multiple views are required to communicate with different stakeholders:

- **Logical View**
- **Development View**
- **Process View**
- **Physical (Deployment) View:** Shows servers, nodes, & network.
- **User View**

Styles & Patterns

- **Architectural Styles:** High-level templates for organizing systems (e.g., Distributed Systems).
- **Architectural Patterns:** Reusable solutions to recurring architectural problems, providing more specific guidance than styles.

Design Synchronicity

This is the measurement of how well the final implementation reflects the architectural & detailed design. High synchronicity is required for successful implementation, particularly in multi-year efforts where deviation is common. This requires a well-defined process of documentation, review, & approval for all changes.

Study Guide

This study guide provides a comprehensive review of software architecture fundamentals, key architectural tasks, & the requirements engineering process. It is designed to synthesize the core principles of high-level system design & the systematic approach to translating requirements into architectural products.

Part I: Knowledge Review Quiz

1. What is the difference between software architecture as a “process” & as a “product”?

The process of software architecture involves the foundational design activity of evaluating & translating requirements into structural & behavioral design elements. The product is the resulting output of this activity, typically manifested as high-level diagrams, component models, & blueprints that specify how major components interact.

2. Explain the primary distinction between Software Architecture & Detailed Design regarding interaction levels.

Software Architecture focuses on high-level interaction, specifically how major subsystems or components (like a frontend & a database) connect & communicate. In contrast, Detailed Design focuses on low-level interaction, defining the internal implementation of those components through specific classes, methods, data structures, & algorithms.

3. Why is it necessary to use a collection of diagrams rather than a single structure to describe software architecture?

No single diagram can fully capture all aspects of a system, such as its usability, performance, security, & deployment. Multiple structures are required to communicate different views of the system to various stakeholders, such as developers, managers, & customers, who each have unique concerns.

4. How do quality requirements “trickle down” from the architecture to the implementation phase?

During the architectural phase, quality requirements are assigned to specific components as “quality tags.” These tags act as guiding specifications for programmers during the detailed design & coding phases, ensuring the final code meets the performance, security, or reliability goals established by the architect.

5. What is “design synchronicity,” & why is it important for successful system implementation?

Design synchronicity measures how accurately the final software implementation reflects the original architectural & detailed designs. Maintaining high synchronicity is essential to ensure the system follows established design rules & fulfills the stakeholders’ initial requirements, especially during multi-year efforts where deviation is common.

6. Define the relationship between a quality attribute & a “tactic.”

A quality attribute represents a high-level goal or desired characteristic of the system, such as “security” or “performance.” A tactic is a specific design decision, such as using encryption or caching, that the architect employs to achieve or control the response of that quality attribute.

7. Describe the three classification categories used for requirements during the analysis phase.

Requirements are classified as functional (what the system does) versus non-functional (how the system behaves), product (qualities of the software) versus process (the methods used for development), & imposed (directly from stakeholders) versus derived (identified by the team to meet higher goals).

8. What is the purpose of “conceptual modeling” in the context of requirements engineering?

Conceptual modeling is used to understand the context of requirements, identify the boundaries of the software system, & visualize how the system interacts with its environment. It often marks the beginning of architectural design because system decomposition is necessary to create effective models.

9. What are the essential characteristics that a requirement must exhibit to be considered high-quality?

High-quality requirements must be specific (clear & concise), consistent (non-conflicting), correct (accurate), attainable (realistic), complete (thorough), & verifiable (testable). If requirements lack these traits, they can lead to incorrect designs, deceptive validation, or impossible implementation goals.

10. How does requirement prioritization assist in the software architecture process?

Prioritization identifies the most critical functions of a system, allowing architects to refine the project schedule by determining which components need to be processed first. This helps in planning different software builds & provides a basis for negotiation when stakeholders' requirements conflict.

Part II: Answer Key

1. **Process vs. Product:** Process is the activity of translating requirements into design elements; Product is the resulting blueprint/diagrams.
2. **Interaction Levels:** Architecture is component-to-component (big picture); Detailed Design is object-to-object/class-to-class (detailed picture).
3. **Multiple Diagrams:** Different stakeholders care about different views, & one diagram cannot represent all dimensions (security, performance, logic).
4. **Trickle Down:** Architects assign quality tags to components, which then serve as requirements for programmers during the coding phase.
5. **Design Synchronicity:** It is the degree of agreement between design & implementation; it ensures the final product follows the intended architectural rules.
6. **Attributes & Tactics:** The quality attribute is the “goal” (what); the tactic is the “design decision” (how) used to reach that goal.
7. **Classification Categories:** Functional/Non-functional, Product/Process, & Imposed/Derived.
8. **Conceptual Modeling:** Used to define system boundaries & interactions; it serves as the bridge between requirements & initial system decomposition.
9. **Requirement Characteristics:** Specific, consistent, correct, attainable, complete, & verifiable.
10. **Prioritization Benefits:** Helps determine the order of development, refines schedules, & aids in conflict resolution during negotiation.

Part III: Essay Format Questions

1. **The Architect's Role in Requirements Engineering:** Discuss why a software architect must be proficient in requirements elicitation & analysis. Address the consequences that occur when an architect is not “immersed” in the requirements phase.
2. **Evaluating the Impact of Early Validation:** Analyze the statement that “fixing defects found earlier on takes much less effort to correct than if found at later stages.” Provide an example using a large-scale system to illustrate the cost of late-stage architectural changes.
3. **The Multi-Dimensional Nature of Software Architecture:** Explain the three primary dimensions of architecture—structure, communication (behavior), & non-functional requirements. How does each dimension contribute to a system that is “difficult to change once built”?

4. **Tactics for System Reliability:** Choose three quality attributes (e.g., Availability, Security, Performance) & explain the specific tactics an architect might use to ensure the system exhibits these traits. Use the “goal vs. decision” framework in your response.

5. **Achieving Design Synchronicity in Long-Term Projects:** Explore the challenges of maintaining high design synchronicity over several years of development & maintenance. Propose policies or processes that an architect could introduce to ensure the implementation remains aligned with the design.

Glossary of Key Terms

Term	Definition
Architectural Pattern	Reusable solutions to recurring architectural problems that guide detailed design decisions.
Architectural Style	A general template or approach for organizing a system's structure, providing a high-level solution shape (e.g., distributed systems).
Architectural View	A representation of a system from a particular perspective to help stakeholders evaluate specific properties like usability or reusability.
Attainable	A characteristic of requirements indicating they can be realistically achieved within constraints like time, budget, & technology.
Availability	A quality attribute referring to the system's uptime & its ability to remain operational.
Conceptual Modeling	The process of discovering system boundaries & how a system interacts with its environment to further identify requirements.
Design Synchronicity	The measurement of how closely a software implementation reflects its intended architectural & detailed design.
Elicitation	The activity of identifying stakeholders & uncovering their needs, wants, & non-functional requirements.
Functional Requirement	A requirement that describes a specific action or service the system must perform (what the system does).
Interoperability	The ability of different software or hardware systems to work together & share data seamlessly.
Modifiability	The degree of complexity involved in changing a system to fit current or future needs.
Non-functional Requirement	A global quality goal describing how well a system should perform (e.g., security, performance).
Portability	The ease with which a system can be adapted to different software or hardware environments.
Requirements Engineering	The systematic approach to requirements specification involving elicitation, analysis, specification, & validation.
Software Architecture	The high-level structures of a software system, the discipline of creating them, & the documentation of these structures.
Software Design	The detailed process of working out the internal logic of components, including classes, functions, & algorithms.
Stakeholder	Any person, group, or organization with a direct or indirect interest in a system, such as engineers, customers, or managers.
Tactic	A specific design decision made to control the response of a quality attribute (e.g., using a watchdog timer for availability).

Testability	The degree of complexity involved in verifying & validating that a system's functions meet their requirements.
Verifiable	A characteristic of a requirement meaning it can be measured or tested to confirm it has been met.

Software Architecture & Quality Attributes

Executive Summary

Software architecture serves as the critical foundation for achieving system quality. While functional requirements define specific behaviors (the “what”), non-functional requirements—also known as Quality Attributes (QA)—specify the criteria for judging the operation of the system as a whole (the “how”). Quality attributes are not achieved by architecture alone but require consistent attention throughout design, implementation, & deployment.

To address the challenges of unstable definitions & overlapping concerns among attributes, architects utilize **Quality Attribute Scenarios**. These scenarios provide a concrete, measurable framework for evaluating system performance & effectiveness. Furthermore, architects employ **Tactics**, which are primitive design techniques intended to control the system’s response to specific stimuli. Systematic architectural design involves seven key categories of decisions, ranging from resource management to technology choice, all aimed at satisfying the diverse needs of stakeholders, including developers, customers, & end-users.

1. Fundamental Concepts of Architecture & Requirements

System requirements are categorized into two primary types:

- **Functional Requirements:** Define specific behaviors or functions, typically stated as “the system shall do [requirements].” These are implemented during the detailed design phase.
- **Non-functional Requirements (Quality Attributes):** Constraints on how a system implements & delivers functionality. These criteria judge the system’s operation as a whole & are detailed within the system architecture.

The Role of Architecture

Architecture provides the necessary foundation for quality, but it is not a self-actualizing solution. Quality attributes must be considered through all phases of development. Architectural documents serve various stakeholders, each with differing priorities:

- **Development Teams:** Prioritize maintainability.
- **Quality Assurance Teams:** Focus on testability.
- **Customers/End-users:** Often focus on usability & availability.

2. Characterizing Quality via Scenarios

Quality attributes often have overlapping concerns. For example, a Denial-of-Service (DoS) attack affects **Availability** (system is down), **Performance** (system is slow), **Security** (breach occurs), & **Usability** (users cannot access services). To resolve this ambiguity, architects use concrete scenarios.

The Six Parts of a Quality Attribute Scenario

A standard scenario is defined by the following elements:

Part	Description
Source of Stimulus	The entity (human, computer system, attacker) that generates the event.
Stimulus	The specific condition or event that arrives at the system (e.g., a crash, a request).
Environment	The conditions under which the stimulus occurs (e.g., normal operation, overload, power outage).
Artifact	The specific part of the system stimulated (e.g., the UI, database, or communication channels).
Response	The activity undertaken by the system after the stimulus arrives.
Response Measure	Measurable criteria used to test the requirement (e.g., response time < 2 seconds, 99.99% uptime).

Template: “Some (**Source**) generates some events (**Stimulus**) that arrives at some (**Artifact**) under some conditions (**Environment**) & must be dealt with (**Response**) in a satisfactory way (**Response Measure**).”

3. Systematic Quality Design Decisions

Architectural design involves making systematic decisions across seven categories:

1. **Allocation of System Responsibilities:** Deciding which parts of the system handle specific tasks (e.g., login vs. payment processing).
2. **Coordination Model:** Defining how different parts of the system communicate & work together.
3. **Data Model:** Identifying data abstractions, their operations, properties, & organization.
4. **Management of Resources:** Allocating memory, processor time, & network bandwidth (e.g., ensuring resources during high traffic).
5. **Mapping among Architectural Elements:** Planning how code modules correspond to runtime elements like processes.
6. **Binding Time Decisions:** Determining when architectural elements are bound (e.g., choosing a language setting at installation versus at runtime).
7. **Choice of Technology:** Selecting specific tools such as Python for logic, PostgreSQL for databases, or AWS for hosting.

4. Analysis of Major Quality Attributes

Availability

Availability is concerned with system uptime & the duration of failures.

Tactics:

- **Fault Detection:** Ping/Echo, Heartbeat, & Exception handling.

- **Recovery-Preparation & Repair:** Voting, Active Redundancy (duplicate systems running in parallel), & Passive Redundancy.
- **Recovery Reintroduction:** Shadow states (keeping a parallel copy of data), Resynchronization, & Rollback.
- **Prevention:** Removal from service, Transactions, & Process Monitors.

Modifiability

Modifiability focuses on the cost of change in terms of time & money.

Tactics:

- **Localize Changes:** Using semantic coherence & generalizing modules to ensure changes do not impact unrelated components.
- **Prevention of Ripple Effect:** Hiding information, maintaining existing interfaces, & using intermediaries.
- **Defer Binding Time:** Delaying decisions until runtime (e.g., using configuration files instead of hard-coding UI themes) to increase flexibility.

Performance

Performance relates to timeliness & the system's ability to respond to events within constraints.

- **Stimulus Types:** Periodic (regular), Sporadic (unpredictable), Bursty (sudden high activity), & Stochastic (random/probabilistic).
- **Response Measures:** Latency (delay), Deadlines, Throughput (work per second), Jitter (inconsistency), Miss Rate, & Data Loss.

Tactics:

- **Resource Dem&:** Increasing computation efficiency & managing event rates.
- **Resource Management:** Introducing concurrency & maintaining multiple copies of data.
- **Resource Arbitration:** Implementing scheduling policies to prioritize tasks.

Security

Security is the ability to resist unauthorized access while providing service to legitimate users.

Tactics:

- **Resisting Attacks:** Authenticating & authorizing users, maintaining confidentiality/integrity, & limiting access.
- **Detecting Attacks:** Utilizing intrusion detection systems.
- **Recovering from Attacks:** Restoring functionality, identifying threat sources, & maintaining audit trails.

Testability

Testability refers to the ease with which software can demonstrate faults.

Tactics:

- **Manage Input/Output:** Separating interfaces from implementation & using record/playback techniques.
- **Internal Monitoring:** Using built-in monitors to observe the system's internal state.
- **Sample Measure:** Achieving 85% path coverage within three hours of completing a code unit.

Usability

Usability measures how easily a user can learn & accomplish tasks.

Tactics:

- **Separate User Interface:** Keeping UI distinct from underlying system operations to allow design changes without altering code.
- **Support User Initiative:** Providing “Cancel,” “Undo,” & “Aggregate” features.
- **Support System Initiative:** Anticipating user needs via User Models (e.g., movie recommendations based on history) or Task Models.

5. Architectural Tactics Summary Table

Quality Attribute	Example Tactics
Security	Resisting Attacks, Detecting Attacks, Authenticating Users
Testability	Separate interface from implementation, Built-in monitors
Modifiability	Localize changes, Encapsulation, Hide information, Defer binding time
Performance	Concurrency, Increase available resources, Caching, Resource arbitration
Availability	Redundancy (Active/Passive), Heartbeat, Shadow State, Rollback

Study Guide

This study guide provides a comprehensive review of software quality attributes, architectural design decisions, & the tactics used to achieve specific system goals based on the provided lecture materials.

Quality Attributes Quiz

1. What is the fundamental difference between functional requirements & quality attributes?

Functional requirements define specific behaviors or functions that a system must perform, typically phrased as “the system shall do.” In contrast, quality attributes (non-functional requirements) specify criteria used to judge the operation of the system as a whole, focusing on how the system implements & delivers its functionality.

2. Why is architecture alone insufficient for achieving quality attributes?

Architecture provides the necessary foundation for achieving quality, but it cannot guarantee it in isolation. Attention must be paid to details during later phases of development, including implementation & deployment, for that foundation to be effective.

3. How do the concerns of different system stakeholders differ regarding quality attributes?

Each stakeholder prioritizes different attributes based on their role; for instance, developers prioritize maintainability while quality assurance teams focus on testability. End users, conversely, are more likely to be interested in attributes like usability & performance.

4. What is a “Quality Attribute Scenario” & why is it used?

A quality attribute scenario is a concrete means of characterizing a quality attribute to avoid vague terms or overlapping concerns. It provides a testable framework by defining a specific stimulus, the system’s response, & a measurable outcome to judge success.

5. List the six parts of a quality attribute scenario.

The six parts are the Source (the entity triggering the action), the Stimulus (the event that happens), the Environment (the conditions during the event), the Artifact (the part of the system affected), the Response (the activity undertaken by the system), & the Response Measure (the measurable criteria for testing success).

6. What are “Tactics” in the context of architectural design?

Tactics are a collection of primitive design techniques that architects use to achieve a specific quality attribute response. Much like design patterns, they are proven techniques captured & applied by experienced architects to control responses to stimuli.

7. Explain the “Binding Time” design decision.

Binding time decisions determine exactly when architectural elements are chosen or set, such as at design time, compile time, or runtime. Delaying these decisions until runtime (Deferred Binding) increases system flexibility by allowing changes like language settings or UI themes without recompiling the code.

8. Define the “Modifiability” quality attribute & its associated costs.

Modifiability is the degree of complexity involved in changing a system to fit current or future needs. It is primarily concerned with the cost of change, which is measured in terms of time, money, & the effort required for implementation & testing.

8. In performance scenarios, what is the difference between “Latency” & “Throughput”?

Latency refers to the delay before data starts moving or the time taken before a response begins, such as waiting two seconds for a video to play. Throughput measures how much work or data the system processes per second, such as a server handling 1,000 requests per minute.

10. How does “System Initiative” differ from “User Initiative” in usability tactics?

User initiative involves tools that help users manage their own actions, such as “Undo” or “Cancel” buttons. System initiative occurs when the system anticipates & reacts to user needs automatically, such as a streaming service recommending movies based on viewing history.

Answer Key

1. **Functional vs. Non-functional:** Functional requirements state what the system must do, while quality attributes specify how the system performs those actions. The former is handled in detailed design, while the latter is detailed in the architecture.
2. **Architecture Foundation:** Architecture sets the stage, but quality must be considered throughout design, implementation, & deployment to be successfully realized.
3. **Stakeholder Interests:** Developers care about maintainability; testers care about testability; end users care about usability. Priorities vary based on how the stakeholder interacts with the system.
4. **Scenarios & Clarity:** Scenarios solve the problem of unstable definitions. They turn vague requirements into clear, testable, & non-overlapping descriptions of system behavior.
5. **Scenario Components:** The components are Source, Stimulus, Environment, Artifact, Response, & Response Measure. These six steps ensure a comprehensive description of any quality requirement.
6. **Design Tactics:** Tactics are architectural design primitives used to control the system’s response to a stimulus. They are techniques used to reach specific quality goals, like using caching for performance.
7. **Binding Time:** This involves determining when architectural elements are bound. Deferred binding (runtime) is often preferred for ease of modification & increased flexibility.
8. **Modifiability Cost:** Modifiability measures how easy it is to change the system. Success is evaluated by the cost in effort, money, & time, as well as the complexity of the affected artifacts.
9. **Latency vs. Throughput:** Latency is about response delay (time), while throughput is about the volume of work (capacity) handled within a specific timeframe.
10. **Usability Initiative:** User initiative empowers the user to control or revert actions. System initiative uses models (User, System, or Task models) to predict what the user needs.

Essay Questions

1. **Stakeholder Influence on Architecture:** Discuss how the conflicting priorities of different stakeholders (e.g., customers versus developers) influence the architectural design process. How does an architect balance these competing quality attribute requirements?

2. **The Role of Scenarios in Resolving Attribute Overlap:** Using the example of a Denial-of-Service (DoS) attack, explain how quality attribute scenarios help distinguish between concerns of availability, performance, security, & usability.
3. **Evaluating Architectural Tactics:** Compare the tactics used for “Fault Detection” in Availability with “Detecting Attacks” in Security. How do these technical decisions impact the overall system response?
4. **The Impact of Binding Time on System Lifecycle:** Analyze the advantages & disadvantages of deferred binding time decisions. Under what circumstances would an architect choose hard binding at compile time over deferred binding at runtime?
5. **Performance & Resource Management:** Explain the relationship between Resource Dem&, Resource Management, & Resource Arbitration. How do these tactics work together to ensure a system meets its performance deadlines & throughput goals?

Glossary of Key Terms

Term	Definition
Availability	The degree to which a system is operational & accessible when required; concerned with system failure & the duration of those failures (uptime).
Binding Time	Decisions that determine when architectural elements are set or chosen (e.g., design time, compile time, or runtime).
Caching	A tactic to improve response time by storing frequently accessed data in a temporary storage area.
Functional Requirements	Requirements that define the specific behavior or functions the system must perform (the “what”).
Interoperability	The system’s ability to collaborate & exchange information with other software or hardware systems.
Jitter	The inconsistency in response times within a system.
Modifiability	The degree of complexity & cost (time/money) involved in changing the system to meet new requirements.
Performance	The system’s capacity to accomplish work under time & resource constraints; focuses on timeliness.
Portability	The degree of complexity involved when adapting a system to different software or hardware environments.
Quality Attribute (QA)	Non-functional requirements that specify criteria for judging the operation of a system (the “how”).
Redundancy	A tactic used to prevent system failure by implementing multiple instances of critical components (e.g., active or passive redundancy).
Reliability	A measure of the system’s failure rate over time.
Response Measure	The measurable criteria used to judge the success of a system’s response to a stimulus (e.g., response time < 2 seconds).
Security	The system’s ability to resist unauthorized access while providing service to legitimate users; involves resisting, detecting, & recovering from attacks.
Stimulus	A condition or event that arrives at the system & requires a response.
Tactics	Primitive design techniques used by architects to achieve specific quality attribute responses.
Testability	The ease with which a system demonstrates its faults through verification & validation.
Usability	How easy it is for users to learn the system, accomplish tasks, & the support the system provides for those tasks.
User Neutrality	A principle ensuring the system & its documentation focus entirely on the content & data rather than the specific user.

Software Architecture Styles & Patterns

Executive Summary

Software architecture is the fundamental organization of a system, decomposed into logical components to satisfy functional & quality requirements. The transition from “scratch-built” designs to the use of established **Architectural Styles & Architectural Patterns** represents a shift toward professional reuse & efficiency.

Key takeaways include:

- **Definitions:** Architectural Styles define the high-level structure (the “big picture”), while Architectural Patterns provide detailed, reusable recipes for specific problems within that structure.
- **Historical Foundation:** Modern software patterns evolved from physical architecture (Christopher Alexander) into a hierarchical classification of system-wide architectural patterns & medium-scale design patterns.
- **Classification:** Systems are categorized by their primary characteristics: Data-Centered, Data Flow, Distributed, Interactive, or Hierarchical.
- **Value of Reuse:** Implementing established patterns benefits from documented experience, highlighting solution approaches, benefits, & consequences. This leads to improved quality, consistency across projects, & increased design efficiency.

Defining Architectural Styles & Patterns

The architectural integrity of a software system depends on the strategic use of past experience. The following distinction is critical for architects:

- **Architectural Style:** A high-level template that defines the overall structure of a system. Example: Layered Style.
- **Architectural Pattern:** A specific, reusable solution to a common problem within a structure. Example: Model-View-Controller (MVC) within a layered style.

The Benefits of Pattern-Based Design

Using well-understood & tested patterns provides three primary advantages:

1. **Improved Quality:** Patterns are based on best practices refined over time, resulting in more robust & reliable software.
2. **Consistency:** Uniformity in architecture across different projects or application segments makes systems easier to maintain.
3. **Increased Efficiency:** Utilizing documented solutions speeds up the design & implementation phases.

The Evolution of Architectural Documentation

The history of software architecture is a progression from physical design principles to specific software-centric solutions.

Era	Figure/Entity	Contribution
1977	Christopher Alex&er	Created a catalogue of 253 patterns for buildings; established the language of patterns (problem + solution + context).
1990s	SE Community	Introduced Architectural Styles for high-level system structures (e.g., Client-Server).
1994	Gang of Four (GoF)	Published 23 Design Patterns focused on object-oriented solutions at the class/object level.
1996	Buschmann et al.	Unified the field by defining Architectural Patterns (system-wide) vs. Design Patterns (subsystem-level).

Classification of Architectural Systems

Architectural patterns are selected based on the system type, requirements, & desired quality attributes.

Type	Description	Common Patterns
Data-Centered	Centralized repository for data; clients access & perform work on that data.	Blackboard, Repository
Data Flow	Oriented around the transport & transformation of a data stream.	Pipes-&-Filters
Distributed	Interaction between independent processing units over a network.	Client-Server, Broker
Interactive	User-centric systems requiring heavy user-system interaction.	MVC
Hierarchical	Components structured in layers or hierarchies to reflect levels of abstraction.	Layered

Detailed Analysis: Data-Centered Systems

Data-centered systems are decomposed around a central data repository. Communication is characterized by bidirectional interaction between “worker” components & a central “data management” component. Workers do not interact directly with each other.

The Blackboard Architectural Pattern

This pattern is designed for complex, “opportunistic” problem-solving where there is no predetermined sequence of operations. It is commonly used in expert systems, speech recognition, & image recognition.

Core Components:

- **The Blackboard:** A shared knowledge base (central hub) where all current information is displayed & accessible via an API.

- **Agents:** Specialized problem solvers (independent contributors) with unique expertise. They monitor the blackboard & jump in to contribute partial solutions when their expertise is relevant.
- **Controller:** An orchestrator that manages which agent accesses the blackboard at any given time, ensuring an orderly progress toward a solution.

Workflow Example: Student Scheduling System

1. **Client** requests a schedule via a UI.
2. **Controller** (ScheduleManager) invokes the **StudentHistory Agent**.
3. **StudentHistory Agent** reads from the **Blackboard**, modifies the schedule based on degree requirements, & writes back to the Blackboard.
4. **Controller** invokes the **CourseOfferings Agent**, which reads the modified schedule, checks availability, & updates the Blackboard.
5. The process continues until the **Controller** retrieves the finalized schedule from the Blackboard & delivers it to the client.

Detailed Analysis: Data Flow Systems

Data flow systems focus on the sequential transformation of data. The most prominent pattern in this category is **Pipes-&-Filters**.

Pipes-&-Filters Pattern

This pattern involves a series of processing elements (Filters) connected by conduits (Pipes) that transmit data.

Core Components:

- **Data Source:** The producer of the data.
- **Filter:** A component that processes, enhances, or modifies data. It produces an output based on its input.
- **Pipes:** The connections that transmit data from the source to filters, or between filters.
- **Data Sink:** The final consumer of the data.

Example: The Compiler Process

- **Scanner (Filter):** Breaks source code into tokens.
- **Parser (Filter):** Organizes tokens into a parse tree (structural hierarchy).
- **Code Generator (Filter):** Translates the parse tree into machine code or assembly instructions.

Distributed & Interactive Systems

While these categories can overlap, their primary focus differs based on connectivity & machine locality.

Interactive (Local) Systems

These systems run fully on a single machine & are designed for direct user interaction.

- **Examples:** Desktop calculator, offline dictionary, offline games.
- **Logic:** All processing & data management occur locally.

Distributed (Networked) Systems

These involve interactions between independent units over a network, supporting multiple users simultaneously.

- **Client-Server Pattern:** Multiple clients request services from a central server (e.g., Web applications, Banking apps).
- **Broker Pattern:** A mediator component coordinates communication between different processes, often used in Service-Oriented Architectures (SOA) to handle routing & message translation.

Hierarchical Systems

Hierarchical systems structure components in layers to reflect different levels of responsibility & abstraction.

- **Vertical Structure (Layered):** Each layer has a specific role (e.g., hardware interaction at the bottom, UI at the top).
- **Separation of Concerns:** Components are grouped based on responsibility, preventing high-level logic from being entangled with low-level hardware operations.
- **Example:** Operating systems & complex control systems in manufacturing or robotics.

Summary Checklist for Pattern Selection

- **Complex Problem/No Fixed Order:** Use **Blackboard**.
- **Sequential Transformation:** Use **Pipes-&-Filters**.
- **Networked Request/Response:** Use **Client-Server**.
- **Cross-Network Coordination:** Use **Broker**.
- **Abstraction Layers:** Use **Hierarchical/Layered**.

Study Guide

This study guide provides a comprehensive review of software architectural styles, patterns, & their historical evolution as outlined in the provided lecture materials. It includes a short-answer quiz, essay prompts, & a glossary of essential terms.

Part I: Short-Answer Quiz

Instructions: Answer the following questions in 2-3 sentences based on the provided source context.

1. **What is the fundamental difference between an architectural style & an architectural pattern?**

An architectural style serves as a high-level template defining the overall structure of a system, such as a Layered style. In contrast, an architectural pattern provides a specific, reusable solution to a common problem within that established structure, such as the MVC pattern used within a Layered style.

2. According to the lecture outlines, what are the primary benefits of reusing architectural styles & patterns?

Reusing these established strategies improves software quality by applying refined best practices & ensures consistency across different projects or application components. Additionally, it increases efficiency by speeding up the design & implementation phases through well-understood & tested solutions.

3. How does Christopher Alex&er’s work in 1977 relate to modern software architecture?

Alex&er originally created a language & a catalogue of 253 patterns for physical buildings to help design quality structures. His work introduced the “language of patterns”—documenting problems, contexts, & solutions—which later inspired the software engineering community to create similar catalogues for digital systems.

4. In the context of mobile applications, when is a system considered “Interactive” versus “Distributed”?

A mobile application is considered an Interactive system if it runs fully on one machine & works offline, such as a calculator or an offline dictionary. It is classified as a Distributed system if it requires network communication to interact with a server, such as Gmail or a banking app.

5. What is the specific role of the “Controller” within the Blackboard architectural pattern?

The Controller acts as an organizer or mediator that manages & coordinates when & which agents have access to the blackboard. It decides the order of problem-solving & triggers agents to run their algorithms based on the current state of information stored in the shared workspace.

6. How does the “Pipes-&Filters” pattern facilitate data transformation?

In this pattern, data flows through a chain of processing elements called filters, which are connected by pipes that transmit the data. Each filter independently processes, enhances, or modifies the input data it receives before passing the transformed output to the next stage in the sequence.

7. Define the “opportunistic problem-solving approach” used in Blackboard systems.

This approach means that independent components (agents) work on partial solutions to a complex problem without a predetermined or fixed sequence of operations. Agents “jump in” to contribute based on the current information available on the blackboard, advancing the solution as they identify opportunities to apply their specific expertise.

8. What are the three main components of a Data-Centered system?

Data-Centered systems are primarily composed of a main central repository of data, a data management component that controls access, & worker components. The worker components execute operations based

on the data but do not interact with each other directly; they communicate only through the central data manager.

9. How did Buschmann et al. (1996) distinguish between architectural patterns & design patterns?

Buschmann et al. defined architectural patterns as “big picture” solutions that specify system-wide structural properties & affect all subsystems. Design patterns are “medium-scale” solutions that influence the internal architecture of a subsystem but do not change the fundamental structure of the entire software system.

10. What is “Data Integrity,” & why is it a critical concern for Data-Centered architectures?

Data integrity ensures that data remains complete, accurate, & consistent across various systems or during operations like storage & retrieval. It is critical in Data-Centered architectures because multiple worker components rely on a single central repository, making accurate data essential for the entire system’s functionality.

Part II: Quiz Answer Key

1. **Difference:** Styles set the “big picture” template; patterns provide the “detailed recipe” within that frame.
2. **Benefits:** Improved quality, architectural consistency, & increased design/implementation efficiency.
3. **Alex&er’s Influence:** Introduced pattern documentation (context-problem-solution) for buildings, which was later adapted for software.
4. **Interactive vs. Distributed:** Interactive systems run locally/offline; Distributed systems require a network to connect clients & servers.
5. **Controller Role:** Orchestrates agent activity & manages access to the shared blackboard; it does not store or pass data itself.
6. **Pipes-&Filters Transformation:** Filters process or modify data, while pipes act as the connectors between these processing stages.
7. **Opportunistic Approach:** Components contribute independently based on the current state of the problem rather than following a strict, pre-defined order.
8. **Components:** Central repository, data management component, & independent worker components.
9. **Architectural vs. Design Patterns:** Architectural patterns are system-wide/large-scale; Design patterns are subsystem-specific/medium-scale.
10. **Data Integrity:** Refers to the accuracy & consistency of data; vital because the central repository is the single source of truth for all components.

Part III: Essay Format Questions

1. **The Evolution of Pattern Theory:** Trace the historical development of architectural patterns from Christopher Alex&er’s 1977 work on physical architecture to the 1996 integration of styles & patterns by Buschmann et al. Explain how the focus shifted from physical structures to high-level software styles & finally to detailed object-oriented design patterns.
2. **The Blackboard Pattern in Expert Systems:** Discuss why the Blackboard architectural pattern is uniquely suited for expert systems like speech recognition or medical emergency room management.

Analyze the interaction between the blackboard, the agents, & the controller in these complex environments.

3. **Comparative Analysis of System Classifications:** Compare & contrast Data-Centered, Data Flow, Distributed, Interactive, & Hierarchical systems. For each, describe a real-world scenario where that specific classification would be the most appropriate choice for an architect.

4. **Internal Dynamics of the Students' Scheduling System:** Using the provided Blackboard pattern example, explain the step-by-step flow of how a draft schedule is built. Detail the specific contributions of the StudentHistory, CourseOfferings, & WorkSchedule agents & how they use the Blackboard's provider interfaces.

5. **Structural Integrity & Quality Requirements:** Explain how decomposing a software system into logical components helps address both functional & non-functional requirements. Use the concepts of modifiability, reusability, & maintainability to support your argument, specifically referencing the Blackboard pattern.

Glossary of Key Terms

Term	Definition
Architectural Style	A high-level template defining the overall structure of a system (e.g., Layered, Data Flow).
Architectural Pattern	A specific, reusable solution to a common problem within a system structure (e.g., MVC, Client-Server).
Agent	An independent, specialized problem-solver component in a Blackboard system that contributes expertise to a shared solution.
Blackboard	A central data repository or shared workspace where independent agents read, write, & update information to solve complex problems.
Broker	A pattern where a mediator component coordinates communication, routes requests, & translates messages between networked components.
Client-Server	A distributed pattern where multiple users (clients) request resources or services from a central provider (server).
Controller	The component in a Blackboard system that orchestrates agent activity & manages access to the central data store.
Data Integrity	The quality property ensuring that data is complete, accurate, & consistent throughout its lifecycle & across systems.
Data Sink	The final consumer of data in a Pipes-&-Filters architectural pattern.
Data Source	The component that produces the initial stream of data in a Data Flow system.
Design Pattern	Medium-scale solutions (e.g., Observer, Factory) that influence subsystem architecture without changing the system's overall structure.
Filter	A component in a Pipes-&-Filters system that transforms, modifies, or enhances data received from a pipe.
Gang of Four (GoF)	Authors Gamma, Helm, Johnson, & Vlissides, who published the influential 1994 catalogue of 23 object-oriented design patterns.
Hierarchical System	A system structured in layers to reflect different levels of abstraction & responsibility (e.g., Operating Systems).
Pipe	The connector in a Data Flow system that transmits a stream of data from one filter to another.

Principles & Practices of Software Detailed Design

Executive Summary

Detailed design serves as the essential bridge between high-level software architecture & the actual construction of code. It involves the decomposition & refinement of system components into fine-grained elements, including functions, data variables, & internal logical structures. The primary objective of this phase is to produce a design sufficiently complete for implementation while maintaining “synchronicity”—the alignment between architecture, design, & code. Success in detailed design is governed by the application of SOLID principles (particularly SRP, OCP, & LSP), rigorous technical reviews, & comprehensive documentation via the Software Design Document (SDD).

Overview of Detailed Design

Detailed design is the process of refining & expanding the preliminary software architecture to the extent that it is sufficiently complete to be implemented. This stage marks a transition from a macro-design approach to a micro-design approach.

The Design Hierarchy

The software development process follows a logical progression where detailed design occupies a central role:

1. **Requirements Phase:** Gathering & documenting system requirements (e.g., a banking app must allow balance checks).
2. **Architecture (High-Level Design):** Defining the overall structure & interactions (e.g., choosing a client-server model).
3. **Detailed Design (Refinement):** Breaking architecture into smaller components & functions (e.g., specifying password validation & session management).
4. **Construction Phase (Implementation):** Translating the blueprints into executable source code.

The Designer’s Mental Model

Effective designers must operate in two directions:

- **Design to Code:** Moving from requirements & architecture toward implementation.
- **Reverse Engineering:** The ability to analyze existing code to recreate the underlying detailed design & architecture.

Key Tasks in Detailed Design

The detailed design activity is comprised of five major tasks:

1. **Understanding Architecture & Requirements:** Designers focus on the specific requirements allocated to their assigned components rather than the entire system.

2. **Creating Detailed Designs:** This involves both structural & behavioral designs, focusing on:
 - Internal & External Interface Design.
 - Graphical User Interface (GUI) Design.
 - Internal Component Design (Structural & Behavioral).
 - Data Design (Databases & data dictionaries).
3. **Evaluating Detailed Designs:** Primarily conducted through Technical Reviews.
4. **Documenting Software Design:** Capturing all design decisions in the Software Design Document (SDD).
5. **Monitoring & Controlling Implementation:** Ensuring the final code adheres to the design.

Component Design in Object-Oriented (OO) Systems

Component-level design defines the internal logical structure & behavior of the components identified during the architectural phase. In OO systems, this involves a shift from abstract UML components to concrete classes, interfaces, & types.

Structural vs. Behavioral Modeling

- **Structural Design:** Typically modeled using UML class diagrams to define internal data structures.
- **Behavioral Design:** Modeled using sequence diagrams to define how components react & interact in different scenarios.

Abstract Classes & Interfaces

Object-oriented designers must distinguish between these two mechanisms for abstraction:

Feature	Abstract Class	Interface
Methods	Can have both abstract (no body) & concrete (with body) methods.	Originally only abstract; (Java 8+ allows default/static methods).
Variables	Can have instance variables.	Only public static final (constants).
Inheritance	A class can extend only one abstract class.	A class can implement multiple interfaces.
Constructor	Can have a constructor.	Cannot have constructors.
Use Case	Used when classes share common state & behavior.	Used to define a “contract” or blueprint for behavior.
Coupling	Tighter; carries some implementation.	Looser; only defines method signatures.

The SOLID Principles of Internal Design

To manage complexity & ensure reusability, designers follow the SOLID principles. The provided context details the first three:

1. Single Responsibility Principle (SRP)

SRP states that a class should have only one reason to change, meaning it should perform only one job.

- **Core Concept:** A module should be responsible to only one user or stakeholder.
- **Benefits:** This keeps designs maintainable & flexible. If a class handles storing data, calculating grades, & printing reports, a change in the database format forces a change in the class, potentially breaking unrelated printing logic. Separating these into StudentData, GradeCalculator, & ReportPrinter mitigates this risk.

2. Open-Closed Principle (OCP)

OCP dictates that software designs should be open to extension but closed for modification.

- **Core Concept:** New functionality should be added by writing new code (extensions) rather than changing existing, working code.
- **Mechanism:** This is achieved by identifying areas likely to vary & encapsulating them through polymorphism. For example, a GameEngine should depend on an abstract Character class rather than a concrete TerrestrialCharacter, allowing new character types to be added without modifying the engine.

3. Liskov Substitution Principle (LSP)

LSP requires that objects should be replaceable with instances of their subtypes without altering the correctness of the program.

- **The Signature Rule:** Method signatures in the subtype must match the base class to ensure type correctness.
- **The Methods Rule:** Overridden methods must follow the base class contract. Subtype methods can “weaken” pre-conditions (require less) but cannot “strengthen” them (require more).
- **Example:** A Square that inherits from Rectangle might violate LSP if the user expects to set height & width independently, as changing one in a square necessarily changes the other.

Evaluation & Documentation Standards

Technical Reviews

A successful evaluation involves a formal review process with the following criteria:

- **Timing:** Provide review notices early to allow thorough preparation.
- **Team Composition:** Include technical experts, stakeholders, & members of the Quality Assurance/testing teams.
- **Focus:** Center the review on how the design meets functional & non-functional requirements.
- **Record-Keeping:** Document the process & assign action items for any identified issues.

The Software Design Document (SDD)

The SDD is the primary reference for programmers, testers, & maintainers. Its standard structure includes:

- **Introduction:** Date of issue, scope, authorship, & summary.
- **Software Architecture:** Overview, stakeholders, & various architectural viewpoints.
- **Detailed Design:** Component-specific design viewpoints & detailed internal views.
- **Glossary & References:** Definitions of terms & lists of applicable documents or technologies used.

Management & Implementation

A critical aspect of detailed design is **Synchronicity**, which measures how well the detailed design adheres to the architecture & how well the code adheres to the design. Low synchronicity is a significant indicator of potential project failure. Maintaining high synchronicity is especially vital during the maintenance phase or when new engineers join a project.

Fundamental Rules of Design

- **What before How:** Define functionalities at every level of abstraction before deciding on the structures to implement them.
- **Simple is Better:** Fancy designs are often buggier & harder to verify. Complex problems should be broken down into simpler, individual problems.
- **Design for Extensibility:** A good design is “open-ended” & avoids restricting irrelevant details or introducing immaterial elements.
- **External before Internal:** Consider the solution as a “black-box” to decide how it interacts with its environment before organizing its internal components.

Study Guide

This study guide provides a comprehensive overview of the principles, tasks, & methodologies associated with the detailed design phase of software development. It synthesizes information regarding the transition from high-level architecture to implementable code, the application of object-oriented design principles, & the documentation necessary for successful software construction.

Part 1: Comprehensive Review

Overview of Detailed Design

Detailed design is the process of refining & expanding the preliminary software architecture of a system or component until it is sufficiently complete for implementation. It serves as a critical bridge between high-level design (architecture) & the actual source code.

- **Placement in the Lifecycle:** It follows the requirements phase (defining what the system must do) & the architecture phase (defining overall structure). It precedes the construction phase (coding).

- **Focus:** Designers delve deep into each component to define internal structures & behavioral capabilities.
- **Macro to Micro:** Design efforts shift from a macro-level approach to a micro-level approach, decomposing system components into fine-grained elements, functions, & data variables.

Key Tasks in Detailed Design

The detailed design activity involves five major tasks:

1. **Understanding Architecture & Requirements:** Unlike architecture, which looks at the complete set of requirements, detailed design focuses on requirements allocated to specific components.
2. **Creating Detailed Designs:** This involves both structural & behavioral designs, including interface design (internal/external), GUI design, internal component design, & data design (databases & data dictionaries).
3. **Evaluating Detailed Designs:** Primarily conducted through Technical Reviews involving experts, stakeholders, & quality assurance teams.
4. **Documenting Software Design:** Capturing details in the Software Design Document (SDD).
5. **Monitoring & Controlling Implementation:** Ensuring the final code adheres to the design.

Documentation & Evaluation

The **Software Design Document (SDD)** is the primary artifact, used by programmers, testers, & maintainers. Its standard sections typically include:

- **Scope:** High-level overview & objectives.
- **References:** List of applicable documents & technologies.
- **Body:** The main section where the design is documented for construction.
- **Glossary:** Definitions of terms & acronyms.
- **Interface Control Document (ICD):** Describes system interfaces & component communication.
- **Version Control Document:** Contains release information, including files, scripts, & executables.

Technical Reviews are the preferred evaluation technique. They require early notice for participants, the inclusion of technical experts & stakeholders, & a focus on how the design meets functional & non-functional requirements.

Implementation Management & Synchronicity

Detailed Design Synchronicity refers to the degree to which:

- Detailed designs adhere to the software architecture.
- Software code adheres to the detailed design. Low synchronicity is a flaw that can lead to project failure, particularly during the maintenance phase or when new engineers join a project.

Component-Level Design

Component design defines the internal logical structure & behavior of the components identified during the architecture phase. In Object-Oriented (OO) systems, this is modeled using UML class diagrams for structure & sequence diagrams for behavior.

Object-Oriented Foundations

To succeed in component design, designers must be proficient in:

- **Classes & Objects:** The basic building blocks of OO design.
- **Abstract Classes:** Classes that cannot be instantiated & contain at least one abstract method (no body). They serve as blueprints for concrete subclasses & promote code reusability.
- **Interfaces:** Blueprints that contain only abstract methods (though Java 8+ allows default/static methods with bodies). They allow for loose coupling & multiple inheritance of type.
- **Inheritance & Polymorphism:** Mechanisms that allow for generalization & dynamic method resolution at runtime.

SOLID Principles (Initial Concepts)

Three core SOLID principles are highlighted for internal component design:

1. **Single Responsibility Principle (SRP):** A class should have only one reason to change, meaning it should have only one job or responsibility. This makes the system more maintainable & flexible.
2. **Open-Closed Principle (OCP):** Software designs should be open for extension but closed for modification. Enhancements should be made by adding new code (often through polymorphism) rather than changing existing, working code.
3. **Liskov Substitution Principle (LSP):** Subtypes must be substitutable for their base types without altering the correctness of the program. This requires maintaining method signatures & honoring the behavioral contract of the supertype.

Part 2: Short-Answer Quiz

Instructions: Answer the following questions in 2-3 sentences based on the provided materials.

1. What is the primary goal of the detailed design phase?
2. How does the “Mental Model” of a designer work in both directions?
3. What is the difference between an Interface Control Document & a Version Control Document?
4. Define “Detailed Design Synchronicity” & why it is important.

5. What are the requirements for a class to be considered an “Abstract Class”?
6. Why are interfaces useful for achieving “loose coupling”?
7. Explain the “Real-Life Analogy” of the Single Responsibility Principle (SRP) provided in the text.
8. What does it mean for code to be “Closed for Modification” under the OCP?
9. Describe the “Signature Rule” within the Liskov Substitution Principle (LSP).
10. According to the “Rules of Design,” why should external functionality be designed before internal functionality?

Part 3: Answer Key

1. **Goal:** The goal is to refine & exp& the software architecture to the point that the design is sufficiently complete for implementation. It ensures that internal structures & behaviors are defined well enough for natural & efficient software construction.
2. **Mental Model:** Designers must be able to move from requirements & architecture forward into detailed code (forward engineering). Conversely, they must be able to analyze existing code to recreate the detailed design & architecture (reverse engineering).
3. **Documentation:** An Interface Control Document describes the system’s interfaces & how components communicate. A Version Control Document lists what is included in a specific software release, such as files, scripts, & executables.
4. **Synchronicity:** This measures how well the detailed design follows the architecture & how well the code follows the design. High synchronicity is vital to prevent project failure, especially during maintenance or team transitions.
5. **Abstract Class:** An abstract class must have the abstract keyword in its declaration & should contain at least one abstract method (a method without a body). It cannot be instantiated & serves as a template for subclasses which must implement its abstract methods.
6. **Loose Coupling:** Interfaces allow code to depend on contracts (method signatures) rather than concrete implementations. This makes the system more flexible because the underlying implementation can change without affecting the components that rely on the interface.
7. **SRP Analogy:** The text uses a restaurant analogy where the chef only cooks, the waiter only serves, & the cashier only h&les money. If one person tries to do all three jobs, the system becomes chaotic; similarly, a class should have only one responsibility to remain maintainable.
8. **OCP (Closed for Modification):** This means that the original, working source code should remain untouched when new features are added. Instead of editing existing code, designers should incorporate new functionality by adding new code & using mechanisms like polymorphism.
9. **LSP Signature Rule:** This rule ensures that if a program is type-correct using a supertype, it remains type-correct when a subtype is used. It requires that method signatures (parameters & return types) in the subtype match or be more general than those in the base class.
10. **External Before Internal:** Designers should first treat a solution as a “black-box” to decide how it interacts with its environment. Only after the external interactions are defined should they decide how the box is internally organized into smaller components.

Part 4: Essay Format Questions

Instructions: These questions are designed for deeper reflection & do not have provided answers.

1. **The Bridge Analogy:** Discuss how detailed design acts as the “bridge” between architecture & code. What are the risks of skipping or rushing this phase?

2. **Technical Review Best Practices:** Elaborate on the importance of including diverse roles (experts, stakeholders, & QA) in a technical review of a detailed design. How does this diversity contribute to the functional success of the system?

3. **Abstract Classes vs. Interfaces:** Compare & contrast the use cases for abstract classes & interfaces. Under what specific design circumstances would you choose one over the other?

4. **The Evolution of Systems via OCP:** Explain how the Open-Closed Principle allows a software system to “evolve gracefully.” Provide a hypothetical example of a system update that follows OCP versus one that violates it.

5. **Complexity & Simplicity:** The “Rules of Design” state that fancy designs are buggier than simple ones. Discuss the designer’s responsibility in identifying “huddled” simple problems within a complex requirement to ensure a maintainable final product.

Glossary of Key Terms

Term	Definition
Abstract Class	A class declared with the abstract keyword that cannot be instantiated & typically contains methods without bodies to be implemented by subclasses.
Component Design	The detailed design task of defining the internal logical structure (classes, types) & behavior (algorithms, communication) of software components.
Detailed Design	The process of refining preliminary software architecture into a complete specification ready for implementation.
Interface	A blueprint for a class that contains only abstract methods (contract), used to achieve abstraction & loose coupling.
Liskov Substitution Principle (LSP)	A SOLID principle stating that objects of a superclass should be replaceable with objects of its subclasses without breaking the application.
Open-Closed Principle (OCP)	A design principle asserting that software entities should be open for extension but closed for modification.
Polymorphism	The ability of different classes to be treated as instances of the same general class through inheritance, allowing for dynamic method resolution.
Reverse Engineering	The process of analyzing existing code to determine the system's structure & behavioral design.
Software Design Document (SDD)	The primary documentation describing a software design, used by developers, testers, & maintainers throughout the lifecycle.
Single Responsibility Principle (SRP)	A principle stating that a class should have one, & only one, reason to change.
Synchronicity	The degree of alignment between architecture & detailed design, & between detailed design & the actual code.
Technical Review	An evaluation technique where a design is scrutinized by a team of experts & stakeholders to ensure it meets requirements.

Detailed Analysis of Creational Design Patterns & the Abstract Factory

Executive Summary

Software design patterns are recurring, reusable solutions to common object-oriented design problems that provide templates for efficient & consistent development. Among these, **Creational Design Patterns** focus on abstracting & controlling the object creation process by specifying common creational interfaces.

The **Abstract Factory Pattern** is a primary example of an object-creational design pattern. Its central purpose is to provide an interface for creating families of related or dependent objects without specifying their concrete classes. By decoupling the client code from concrete implementations, the pattern promotes consistency within product families, facilitates the addition of new product groups through extension (obeying the Open/Closed Principle), & enhances system modifiability. While it significantly improves flexibility & encapsulation, it requires a larger number of classes, which can increase initial design complexity.

Foundations of Design Patterns

Design patterns are distinct from architectural patterns & styles; they are reusable solutions tailored to specific object-oriented contexts to help developers avoid “reinventing the wheel.”

Classification Criteria

The source material classifies design patterns based on two primary criteria: **Purpose & Scope**.

Criterion	Type	Description
Purpose	Creational	Deals with the process of object creation.
	Structural	Deals with the composition of classes or objects to form larger structures.
	Behavioral	Deals with how classes & objects interact & distribute responsibility.
Scope	Class-level	Primarily applies to classes & their relationships during design time.
	Object-level	Primarily applies to objects & their relationships during run-time.

Documenting Patterns

Comprehensive documentation is essential for understanding a pattern’s utility. Key documentation categories include:

- **Intent:** The core purpose of the pattern.
- **Motivation:** The scenario or problem the pattern addresses.
- **Applicability:** When the pattern should be used.
- **Structure:** A visual representation (e.g., UML class diagram).
- **Participants:** The classes & objects involved & their specific responsibilities.

- **Collaborations:** How participants work together.
- **Consequences:** The positive & negative effects on the software solution.
- **Implementation:** Techniques & information for successful deployment.
- **Known Uses:** Real-world examples of the pattern in use.

The Abstract Factory Pattern

The Abstract Factory is characterized as a “one-stop shop” for creating families of related objects while hiding the details of their concrete classes.

Core Intent & Structure

The pattern is intended to manage & encapsulate the creation of a set of objects that conceptually belong together as a specific “family.” Like all creational patterns, it is composed of two primary types of classes:

1. **Creator Classes:** These define the interfaces & concrete implementations for the factories.
2. **Product Classes:** These define the interfaces & concrete implementations for the objects being created.

Key Benefits

- **Encapsulation:** The client is aware of abstract product names (e.g., “Button”) but remains ignorant of concrete class names (e.g., “DarkButton”).
- **Flexibility:** New families can be added by creating a new factory without modifying existing client code.
- **Consistency:** It ensures that products from the same family are used together.
- **Open/Closed Principle (OCP):** Additions are made through extension rather than modification of existing code.

Implementation Methodology

The source outlines a logical seven-step process for designing a system using the Abstract Factory pattern:

1. **Design Product Interfaces:** Define the abstract interfaces for each product type (e.g., CPU, Monitor).
2. **Identify Families:** Determine the different groups required (e.g., Standard vs. Advanced computers).
3. **Design Concrete Products:** For each group, create products that realize the interfaces from Step 1.
4. **Create Factory Interface:** Design an interface containing methods for creating each product interface.
5. **Create Concrete Factories:** Implement a factory for each family identified in Step 2.

6. **Associate Factories & Products:** Link each concrete factory to its specific set of concrete products.
7. **Create the Client:** Develop the client code that interacts only with the product & factory interfaces.

Illustrative Case Studies

1. User Interface (UI) Themes

A program requiring “Dark” & “Light” themes uses an AbstractFactory (e.g., GUIFactory) to create buttons & text fields.

- **Products:** Button & TextField.
- **Concrete Implementations:** DarkButton/DarkTextField & LightButton/LightTextField.
- **Factories:** DarkThemeFactory & LightThemeFactory.
- **Outcome:** The client asks the factory for a Button & does not care if it receives a DarkButton or LightButton. Adding a “Blue Theme” only requires adding a BlueThemeFactory & corresponding products.

2. Computer Store System

A store sells “Advanced” & “St&ard” computers composed of different grades of CPUs, Monitors, & Keyboards.

- **Interface:** ComputerPartsFactory defines methods like createCPU(), createMonitor(), & createKeyboard().
- **Families:** AdvancedComputerPartsFactory produces AdvancedCPU, etc., while St&ardComputerPartsFactory produces St&ardCPU, etc.
- **Data Integration:** Each product implementation can contain specific logic for retrieving data (reviews, comments) from unique remote sources (e.g., Database A for advanced products vs. Database B for st&ard).

3. Automotive Branch Management (RiyadhHQ)

A central client (RiyadhHQ) requests cars from different city branches without needing to know which city produces them.

- **Abstract Factory:** CarFactory with methods makeMercedes() & makeBMW().
- **Concrete Factories:** JeddahCarFactory & DammamCarFactory.
- **Decoupling:** RiyadhHQ only works with the CarFactory interface, allowing the system to switch between city branches at runtime without changing the client’s code.

Evaluative Analysis

Aspect	Details
Pros	Isolation: Concrete product classes are isolated, making them easier to reuse. Consistency: Promotes the use of compatible products. Modifiability: Enhances the ability to update the system. OCP Compliance: New families require no modification to existing code.
Cons	Complexity: A large number of classes are required to implement the pattern. Rigidity in Product Types: Adding a completely new <i>type</i> of product (e.g., adding a “Mouse” to an existing computer factory) requires updating the abstract factory interface & all concrete factory implementations.

Study Guide

This study guide provides a comprehensive review of design patterns in detailed design, with a specific focus on the Abstract Factory creational pattern. It includes a short-answer quiz, essay prompts, & a glossary of key terms based on the provided technical documentation.

Part 1: Short-Answer Quiz

Instructions: Answer the following ten questions based on the provided text. Each answer should be approximately 2–3 sentences in length.

1. What is the fundamental definition & purpose of a design pattern?
2. How are design patterns classified based on their “Purpose”?
3. What is the difference between class-level & object-level pattern scopes?
4. What is the primary function of Creational Design Patterns?
5. Define the specific intent of the Abstract Factory design pattern.
6. What are the two main types of classes that compose the Abstract Factory pattern?
7. How does the Abstract Factory pattern benefit the client regarding concrete class names?
8. According to the documentation, how does the Abstract Factory pattern support the Open-Closed Principle (OCP)?
9. What is the primary disadvantage associated with implementing the Abstract Factory pattern?

10. In the implementation of an Abstract Factory, what is the role of the “Client” class?

Part 2: Answer Key

1. **What is the fundamental definition & purpose of a design pattern?** Design patterns are recurring, reusable solutions to common object-oriented design problems within a particular context. They provide templates that help developers solve problems efficiently & consistently, preventing the need to “reinvent the wheel.”
2. **How are design patterns classified based on their “Purpose”?** Patterns are classified into three purposes: Creational, which deals with object creation; Structural, which focuses on creating structures from existing objects; & Behavioral, which manages class interactions & the assignment of responsibility between objects.
3. **What is the difference between class-level & object-level pattern scopes?** Class-level patterns apply at design time & focus on the relationships between classes. Object-level patterns focus on the interactions & relationships between objects at runtime during program execution.
4. **What is the primary function of Creational Design Patterns?** Creational design patterns abstract & control how objects are created within software applications. They achieve this by specifying a common creational interface that manages the instantiation process.
5. **Define the specific intent of the Abstract Factory design pattern.** The intent of the Abstract Factory is to provide an interface for creating families of related or dependent objects without specifying their concrete classes. It encapsulates the creation of a set of objects that conceptually belong together as a specific family of products.
6. **What are the two main types of classes that compose the Abstract Factory pattern?** Like all creational patterns, the Abstract Factory is composed of creator classes & product classes. In some specific creational patterns, these two roles may be fused into a single class.
7. **How does the Abstract Factory pattern benefit the client regarding concrete class names?** The pattern provides encapsulation where the client knows the abstract product names (such as “Button”) but does not need to know the concrete product class names (such as “DarkButton”). This allows the client to use a factory to get products without being coupled to specific implementations.
8. **According to the documentation, how does the Abstract Factory pattern support the Open-Closed Principle (OCP)?** It supports the OCP because additions of new product families are made through extension rather than modification of existing code. New families can be added by creating a new factory & new product classes without changing the client’s code.
9. **What is the primary disadvantage associated with implementing the Abstract Factory pattern?** The main “con” or disadvantage of using the Abstract Factory pattern is that a large number of classes are required to implement the structure. This includes various interfaces & concrete implementations for every product & factory in the system.
10. **In the implementation of an Abstract Factory, what is the role of the “Client” class?** The Client is the component associated with both the product & factory interfaces. It uses these interfaces to request & interact with objects, which allows it to vary its behavior & work with different product families without internal code changes.

Part 3: Essay Questions

Instructions: Use the provided source context to develop comprehensive responses for the following questions.

1. **Architectural vs. Design Patterns:** Compare & contrast architectural patterns with design patterns as described in the text. Discuss how design patterns function as “templates” for developers.

2. **The Computer Store Example:** Describe how the Abstract Factory pattern would be applied to a computer store system that carries “Advanced” & “Standard” computers. Detail the necessary interfaces & concrete classes for CPUs, Monitors, & Keyboards.
3. **Pattern Scalability:** Explain the process & consequences of adding a new family of products (e.g., a “Blue Theme”) versus adding a new product (e.g., a “Checkbox”) to an existing system using the Abstract Factory pattern.
4. **Decoupling & Modifiability:** Using the RiyadhHQ car factory example, explain why decoupling the client from concrete classes is critical for system modifiability. Discuss how the CarFactory interface facilitates switching between different city-based factories at runtime.
5. **Step-by-Step Implementation:** Outline the seven-step summary for designing a system with an Abstract Factory. Explain the importance of associating concrete factories with their respective products in this workflow.

Glossary of Key Terms

Term	Definition
Abstract Factory	An object-creational design pattern that provides an interface for creating families of related objects without specifying concrete classes.
Behavioral Patterns	Patterns that deal with class interaction, variation of behavior, & the assignment of responsibility between objects.
Class-level Patterns	Design patterns that primarily apply to relationships between classes during design time.
Creational Patterns	Patterns that abstract & control the way objects are created by specifying common creational interfaces.
Design Pattern	A recurring & reusable solution to a common object-oriented design problem in a particular context.
Encapsulation	In the context of Abstract Factory, the practice of hiding concrete product class names from the client, who only knows abstract product names.
Factory Interface	An interface (e.g., GUIFactory) that specifies the methods for creating various abstract products.
Object-level Patterns	Design patterns that primarily focus on the relationships & interactions between objects at runtime.
Open-Closed Principle (OCP)	A design principle stating that software entities should be open for extension but closed for modification; supported by Abstract Factory through the addition of new factories.
Product Classes	The concrete implementations of objects (e.g., DarkButton, StandardCPU) created by the factories.
Structural Patterns	Patterns that deal with the creation of structures to form existing objects.

Structural Design Patterns: Adapter, Composite, & Facade

Executive Summary

Structural design patterns are essential architectural tools that facilitate the design of class & object structures at runtime. This briefing document examines three primary structural patterns: the **Adapter**, the **Composite**, & the **Facade**.

The Adapter pattern enables interoperability between classes with incompatible interfaces by acting as a translator. The Composite pattern allows for the creation of tree-like hierarchies, enabling clients to treat individual objects & collections of objects uniformly. Finally, the Facade pattern provides a simplified, unified interface to a complex subsystem, reducing coupling & complexity for the client. Collectively, these patterns prioritize software reusability, flexibility, & the reduction of client-side complexity through strategic abstraction.

The Adapter Design Pattern

The Adapter pattern is a structural design pattern used to adapt an existing interface into one that a client expects. It serves as a bridge between two incompatible interfaces, allowing classes to work together that otherwise could not.

Intent & Mechanism

The primary intent is to convert the interface of a class into another interface clients expect. The operational flow involves three steps:

1. **Request:** The client makes a request to the adapter by calling a method on it using the target interface.
2. **Translation:** The adapter translates that request into a call that the “Adaptee” (the legacy or incompatible class) understands using the adaptee’s interface.
3. **Result:** The client receives the results of the call seamlessly.

Structural Variations

There are two primary implementations of this pattern:

Feature	Object Adapter	Class Adapter
Mechanism	Uses Composition : The adapter holds an instance of the Adaptee.	Uses Inheritance : The adapter class inherits from the Adaptee class.
Binding Time	Runtime delegation (adaptee.specificOperation()).	Compile-time implementation (specificOperation()).
Language Support	Broadly supported.	Requires multiple inheritance, which is not supported by many languages (e.g., Java).

Preference	Generally preferred due to fewer inheritance-related issues.	Less common due to language limitations.
-------------------	--	--

Case Study: Bird to ToyDuck Adaptation

In a scenario where a system expects ToyDuck objects (Target) but only Bird objects (Adaptee) are available, an adapter is used:

- **Target Interface (ToyDuck):** Expects a squeak() method.
- **Adaptee (Bird/Sparrow):** Contains fly() & makeSound() methods.
- **Adapter (BirdAdapter):** Implements ToyDuck & wraps a Bird instance. When the client calls squeak(), the adapter internally triggers bird.makeSound().

Evaluation

- **Advantages:** Achieves high reusability & flexibility. It allows clients to use polymorphism to swap between different adapter implementations without complicating client code.
- **Disadvantages:** Introduces a slight increase in overhead due to request forwarding. In complex systems, “adapter chains” may be required to reach the desired type, increasing architectural depth.

The Composite Design Pattern

The Composite pattern is an object structural pattern used to represent whole-part hierarchies through tree-like structures. It is uniquely designed to allow clients to treat individual “leaf” objects & “composite” groups of objects uniformly.

Core Components

1. **Component:** The base interface or class that defines common operations for both simple & complex objects.
2. **Leaf:** A primitive object that represents a node with no children. It implements the component’s operations directly.
3. **Composite:** A complex object that stores a collection of child components (leaves or other composites). It implements component operations by delegating to its children.

Implementation Requirements

To implement a Composite structure, a system must:

- Identify the tree-like structure required.
- Design a **Component base class** with overridable methods.
- Design a **Composite class** with an internal data structure (e.g., an ArrayList) to store nodes & methods to add or remove children.
- Design **Leaf classes** that override component methods to perform specific tasks.

Real-Life Applications

Example	Primitive (Leaf)	Composite
File System	File (cannot contain other objects).	Folder (contains files & other folders).
University	Single Course (e.g., Math 101).	Program (e.g., Computer Science degree).

Benefits

- **Uniformity:** Minimizes client complexity by shielding it from the operational differences between primitive & composite objects.
- **Scalability:** Makes it easy to add new component types without altering the existing client code.
- **Structure:** Provides a clear design for systems that naturally follow a whole-part relationship.

The Facade Design Pattern

The Facade pattern provides a simplified, higher-level interface to a set of interfaces in a complex subsystem. It acts as a single point of entry, making the subsystem easier to use.

Intent & Implementation

The Facade defines a unified interface that hides the “noise” & complexity of a subsystem.

Procedure for Application:

1. **Identification:** Determine all components & operations involved in a subsystem task.
2. **Design:** Create a Facade class (e.g., a ShopKeeper) that includes an interface method for the client.
3. **Encapsulation:** Implement the Facade’s method by calling the necessary operations on the internal subsystem components (e.g., Iphone, Samsung, Blackberry).
4. **Access:** Clients interact only with the Facade type to execute subsystem operations.

Case Study: Mobile Shop Subsystem

A client wanting to interact with a mobile phone subsystem (consisting of various br&s & price models) uses a ShopKeeper Facade:

- Instead of the client dealing with Iphone.modelNo(), Iphone.price(), & similar calls for every br&, the client simply calls sk.iphoneSale().
- The ShopKeeper handles the instantiation & specific method calls of the underlying mobile br& classes.

Evaluation of Benefits

- **Reduced Complexity:** Shields clients from the internal workings of complex subsystems.
- **Stability:** The Facade provides a stable interface; even if the internal subsystem changes, the client code remains unaffected.

- **Weak Coupling:** Promotes loose coupling, as clients depend on a single interface rather than multiple, interconnected subsystem components.

Conclusion

The structural patterns detailed in this document offer distinct solutions for managing object relationships. The **Adapter** resolves interface incompatibilities, the **Composite** manages hierarchical part-whole relationships, & the **Facade** simplifies access to complex systems. Implementing these patterns leads to more maintainable, decoupled, & reusable software architectures.

Study Guide

This study guide provides a comprehensive review of structural design patterns as outlined in the lecture materials. Structural patterns focus on how classes & objects are composed to form larger structures, specifically dealing with the design of existing classes or objects at run-time.

Short-Answer Quiz

Instructions: Answer the following questions in 2–3 sentences based on the source context.

1. **What is the core intent of the Adapter design pattern?** The Adapter pattern is designed to convert the interface of a class into another interface that clients expect. This allows classes with incompatible interfaces to work together by acting as a bridge between the client & the existing class.
2. **How does an Object Adapter differ from a Class Adapter in its implementation?** An Object Adapter implements the Target interface by delegating requests to an Adaptee object at run-time using composition. In contrast, a Class Adapter uses inheritance at compile-time to implement the Target interface, though this often requires multiple inheritance support from the programming language.
3. **Explain the three-step process of using an adapter from the client’s perspective.** First, the client makes a request to the adapter by calling a method defined in the target interface. Next, the adapter translates that request into a format the adaptee understands & calls the adaptee’s interface. Finally, the client receives the results of the call from the adapter.
4. **What is a significant disadvantage of the Adapter design pattern?** One drawback is a slight increase in overhead because all requests must be forwarded through the adapter. Additionally, complex systems may sometimes require an “adapter chain” where multiple adaptations are needed to reach the required type.
5. **What does the Composite design pattern allow designers to represent?** The Composite pattern allows designers to create tree-like structures to represent whole-part hierarchies. It enables clients to treat individual objects (leaf nodes) & compositions of objects uniformly through a shared interface.
6. **In the Composite pattern, what is the role of the “Component” base class?** The Component base class defines the interface & includes overridable methods that are common to both Leaf & Composite

objects. This ensures that the client can interact with any object in the tree structure without knowing its specific type.

7. **Describe the internal requirements of a “Composite” class within a tree structure.** A Composite class must override the common methods in the component interface & maintain an internal data structure to store its child nodes. It also includes specific methods to add or remove objects from this internal hierarchy.
8. **What is the primary purpose of the Facade design pattern?** The Facade pattern provides a simplified, higher-level interface to a complex subsystem. Its intent is to define a unified interface that makes a set of interfaces within a subsystem easier for a client to use.
9. **How does the Facade pattern promote “weak coupling”?** It promotes weak coupling because clients only need to depend on a single interface (the Facade) rather than multiple interfaces within a complex subsystem. This shields the client from the internal details & changes of the subsystem.
10. **Using the “ShopKeeper” example, explain how a Facade simplifies a mobile shop subsystem.** In this example, the ShopKeeper class acts as the Facade, providing simple methods like `iphoneSale()` or `samsungSale()`. The client interacts only with the ShopKeeper, remaining unaware of the specific classes like Iphone, Samsung, or Blackberry that implement the MobileShop interface.

Answer Key

1. **Intent:** Convert class interfaces into what clients expect to enable cooperation between incompatible classes.
2. **Implementation:** Object Adapters use delegation/composition at run-time; Class Adapters use inheritance at compile-time.
3. **Process:** Client calls target interface; adapter translates/calls adaptee; client receives results.
4. **Disadvantage:** Forwarding overhead & the potential complexity of adaptation chains.
5. **Representation:** Tree-like structures & whole-part hierarchies where individuals & groups are treated uniformly.
6. **Component Role:** Defines the common interface & overridable methods for both Leaf & Composite objects.
7. **Composite Requirements:** Internal data structure for children, overriding common methods, & add/remove capabilities.
8. **Facade Purpose:** Provide a simplified & unified higher-level interface to complex subsystems.
9. **Weak Coupling:** Minimizes dependencies by having clients interact with one interface instead of many.
10. **ShopKeeper Example:** The ShopKeeper (Facade) hides subsystem details (Iphone/Samsung classes) from the client, offering a simple interface for sales.

Essay Questions

Instructions: Use the provided source materials to develop detailed responses to the following prompts.

1. Compare the **Object Adapter & Class Adapter**. Discuss why the Object Adapter is generally preferred in most modern programming languages, referencing the limitations mentioned in the text.

2. Analyze the **Composite design pattern** using the real-life example of a file system. Explain how files & folders represent “leaf” & “composite” nodes & why uniform treatment is beneficial for a file explorer.
3. Detail the step-by-step procedure for applying the **Facade design pattern** to a subsystem. Explain how each step contributes to shielding the client from internal complexity.
4. Structural patterns deal with “designing the structure of existing classes or objects at run-time.” Discuss how the **Adapter** & **Facade** patterns achieve this, specifically focusing on how they modify or simplify access to existing code.
5. Examine the benefits & drawbacks of the **Adapter pattern**. In what scenarios would the increase in overhead be an acceptable trade-off for the reusability & flexibility provided?

Glossary of Key Terms

Term	Definition
Adaptee	An existing class with an interface that needs adapting to be compatible with a client.
Adapter Pattern	A structural pattern that converts the interface of a class into another interface clients expect.
Class Adapter	A version of the Adapter pattern that uses inheritance to adapt the interface (requires multiple inheritance).
Component	In the Composite pattern, the base class or interface that defines operations common to simple & complex objects.
Composite Pattern	A pattern that composes objects into tree structures to represent whole-part hierarchies.
Facade Pattern	A pattern that provides a simplified, unified interface to a set of interfaces in a subsystem.
Leaf	A primitive object in the Composite pattern that does not have any children.
Object Adapter	A version of the Adapter pattern that uses composition & delegation at run-time to adapt an interface.
Structural Design Patterns	Patterns that deal with the design & composition of existing classes or objects at run-time.
Target Interface	The specific interface that a client requires & depends upon.
Weak Coupling	A design goal where a client depends on a single stable interface (like a Facade) rather than many complex internal components.
Whole-Part Relationship	A hierarchy where a “whole” object is composed of various “parts,” common in tree-like structures.

Software Architecture Evaluation

Executive Summary

Software architecture serves as the critical foundation of any software system, defining its structure, component relationships, & external behaviors. Evaluating this architecture before the construction phase is essential to ensuring a high-quality product & minimizing the costs associated with late-stage design corrections. Evaluation is primarily driven by functional requirements, non-functional requirements, & business constraints.

The Architecture Tradeoff Analysis Method (ATAM) is the most prominent methodology for evaluating architectural decisions. Its primary objective is to assess how architectural choices impact quality attribute requirements & business goals, specifically identifying risks where different quality attributes (such as performance versus security) conflict. By utilizing scenarios & utility trees, ATAM provides a structured approach to prioritizing goals, uncovering cross-project reuse opportunities, & improving documentation. For smaller or less complex projects, a “Lightweight ATAM” offers a more efficient, albeit less rigorous, alternative for internal organizational reviews.

Fundamentals of Architecture Evaluation

Architecture design translates business goals & requirements into a structural blueprint. Because this phase is foundational, early validation is a critical success factor for any software project.

Benefits of Systematic Evaluation

- **Conflict Resolution:** Facilitates the prioritization of conflicting architectural goals.
- **Clarity & Documentation:** Forces a clear explanation of the architecture, leading to higher-quality documentation.
- **Knowledge Reuse:** Uncovers opportunities for reusing architectural patterns across different projects.
- **Process Improvement:** Results in more robust architectural practices over time.

Major Evaluation Methods

While several methods exist, they share a common reliance on scenarios & quality attributes. The three primary methods are:

1. **ATAM (Architecture Tradeoff Analysis Method):** Focuses on quality attributes & business goal trade-offs.
2. **SAAM (Software Architecture Analysis Method):** Used for analyzing architecture through scenarios.
3. **ARID (Active Reviews for Intermediate Designs):** Focused on evaluating design during the intermediate stages.

The Architecture Tradeoff Analysis Method (ATAM)

ATAM is designed to discover risks where architectural decisions affect quality attributes of interest. It recognizes that in complex systems, quality attributes cannot be achieved in isolation; improvements in one area (e.g., security) may negatively impact another (e.g., performance).

Quality Attribute Perspectives

Quality attributes are categorized into three distinct perspectives to ensure comprehensive evaluation:

Perspective	Key Attributes
End-User	Performance, availability, usability, security
Technical	Modifiability, portability, reusability, testability, interoperability
Business Community	Time to market, cost/benefits, projected lifetime, budget

Participant Groups & Roles

A successful ATAM execution requires cooperation between three specific groups:

- **Evaluation Team:** Three to five experienced architects who lead the process.
- **Project Decision Makers:** Individuals with authority to implement changes (e.g., project managers, customers).
- **Architecture Stakeholders:** Individuals whose work depends on the architecture (e.g., developers, testers, users, maintainers).

Specific Team Roles:

- **Team Leader:** Coordinates with the client, establishes contracts, & ensures report delivery.
- **Evaluation Leader:** Runs the evaluation & facilitates scenario elicitation & prioritization.
- **Scenario Scribe:** Captures the exact wording of scenarios on visible media (whiteboards).
- **Proceedings Scribe:** Records the raw scenarios, motivations, & resolutions in electronic form.
- **Questioner:** Raises architectural issues based on expertise in specific quality attributes.

The ATAM Process: Steps & Phases

The ATAM process is structured into four phases spanning nine logical steps.

Execution Phases

Phase	Activity	Participants	Typical Duration
0	Partnership/Preparation: Logistics, planning, & team formation.	Evaluation leadership & key decision-makers.	Informal; several weeks.
1	Evaluation: Steps 1–6.	Evaluation team & decision-makers.	1–2 days; 2–3 week hiatus.
2	Evaluation: Steps 7–9.	Team, decision-makers, & stakeholders.	2 days.
3	Follow-up: Report generation & delivery.	Evaluation team & client.	1 week.

The 9-Step Methodology

1. **Present the ATAM Method:** The evaluation team explains the process, techniques, & expected outputs (risks, sensitivity points, & tradeoffs).
2. **Present Business Drivers:** Decision-makers present business goals, major stakeholders, high-level functional requirements, & constraints (technical, economic, political).
3. **Present Architecture:** The lead architect describes the approach using models like the **4+1 Architectural View Model** (Contextual, Logical, Process, & Deployment views).
4. **Identify Architectural Approaches:** The team identifies patterns used (e.g., client-server, multi-layer, pipes-filters).
5. **Generate Quality Attribute Utility Tree:** A top-down approach to prioritize quality goals. The tree's leaves consist of quality scenarios evaluated by importance & difficulty.
6. **Analyze Architectural Approaches:** The team examines high-ranked scenarios to see how the architecture supports them, documenting rationale, risks, & non-risks.
7. **Brainstorm & Prioritize Scenarios:** Stakeholders brainstorm operationally meaningful scenarios. If these diverge significantly from the architect's utility tree, it identifies a major project risk.
8. **Analyze Architectural Approaches (Refined):** The architect demonstrates how the architecture handles the high-ranked scenarios from the brainstorming session.
9. **Present Results:** The team delivers a report recapping the process, prioritized scenarios, utility tree, & all discovered risks & tradeoffs.

Analysis Artifacts: Risks, Sensitivities, & Tradeoffs

A core output of the ATAM is the identification of critical decision points:

- **Sensitivity Points:** Architectural decisions on which a particular quality attribute is highly dependent.
- **Tradeoffs:** Decisions that affect multiple quality attributes (often improving one while degrading another).
- **Risks & Non-risks:** Decisions that are either problematic or well-grounded in light of the requirements.

Lightweight ATAM

For smaller or less risky projects, a “Lightweight” version of ATAM is available. This version is often conducted by internal staff & can be completed in a single day or half-day.

Key Differences:

- **Efficiency:** Steps like scenario brainstorming (Step 7) & redundant analysis (Step 8) are often omitted or merged into the utility tree generation.

- **Internal Focus:** Participants are usually internal to the organization, which speeds up consensus but may reduce objective depth.
- **Condensed Timeline:** Presentation of business drivers & architecture is brief (15–30 minutes) as participants are already familiar with the project.
- **Main Focus:** The bulk of the time (2–3 hours) is dedicated to Step 6: mapping highly ranked scenarios onto the architecture.

Required Evaluation Outputs

At minimum, an ATAM evaluation must produce:

- **Software Architecture Document (SAD):** The primary artifact specifying the selected architecture.
- **Evaluation Report:** A comprehensive document that recaps the process, captures scenario analysis, explains candidate architectures, & provides the rationale for the final architectural decisions.

Study Guide

This study guide provides a comprehensive review of software architecture evaluation based on the Architecture Tradeoff Analysis Method (ATAM). It explores the drivers of architectural design, the benefits of early evaluation, & the specific procedures involved in both standard & lightweight evaluation processes.

Part 1: Short-Answer Quiz

Instructions: Answer the following questions in two to three sentences based on the source material.

1. **What are the three primary drivers that shape software architecture?** Software architecture is shaped by functional requirements, non-functional requirements, & business constraints. These drivers define the structure of the system, including its components, relationships, & the behaviors exposed to the external world.
2. **Why is it beneficial to evaluate software architecture before the construction phase begins?** Early evaluation is critical because the architecture serves as the foundation for the entire software system. Validating the design in advance allows teams to find & fix errors during the design phase, which is significantly less costly than correcting them after construction has started.
3. **What common elements do the ATAM, SAAM, & ARID evaluation methods share?** While there are several distinct methods for evaluation, ATAM (Architecture Tradeoff Analysis Method), SAAM (Software Architecture Analysis Method), & ARID (Active Reviews for Intermediate Designs) all share a common foundation. Specifically, these methods all utilize scenarios & quality attributes as central components of the evaluation process.

4. **What is the primary purpose of the ATAM method?** The purpose of ATAM is to assess the consequences of architectural decisions by looking at quality attribute requirements & business goals. It aims to discover risks where architectural decisions impact quality attributes, essentially identifying the trade-offs between different system goals.
5. **Describe the three categories of quality attributes used to characterize a system.** Quality attributes are grouped into the end-user perspective (performance, availability, usability, security), the technical perspective (modifiability, portability, reusability, testability, interoperability), & the business community perspective (time to market, cost/benefits, lifetime, budget). These attributes are prioritized using quality attribute scenarios to ensure they align with business goals.
6. **Who are the three main groups of participants required for a successful ATAM process?** A successful ATAM requires an evaluation team of three to five experienced architects & project decision-makers who have the authority to implement changes. Additionally, architecture stakeholders, such as developers, testers, maintainers, & users, must participate to ensure the architecture meets the needs of those who will build & use the system.
7. **Explain the function of a “Utility Tree” in architectural evaluation.** A utility tree is a top-down tool used to identify, prioritize, & refine the most important quality attribute goals. It places high-level nodes like security or performance at the top, while the “leaves” of the tree consist of specific quality scenarios that are evaluated based on their importance to the system’s success & their technical difficulty.
8. **What specific information must be included in an ATAM evaluation report?** The evaluation report must recap the ATAM process & capture the scenario analysis performed during the evaluation. It also provides an explanation of the candidate architectures, the rationale behind the final architectural decisions, & a summary of all work completed.
9. **In Step 3 of the ATAM process, which model is recommended for describing the architecture?** The lead architect is encouraged to use the “4+1” architectural view model to describe the system. This model includes the contextual view, logical view, process view, & deployment view, along with the associated risks for meeting architectural requirements.
10. **How does a “Lightweight ATAM” differ from the standard process in terms of participants & duration?** A Lightweight ATAM is designed for smaller, less risky projects & can be completed in a single day or even a half-day meeting. Unlike the standard method, it is often carried out entirely by internal organization members, which speeds up the process but may offer less depth & objectivity.

Part 2: Answer Key

1. Functional requirements, non-functional requirements, & business constraints.
2. It allows for finding problems early when they are less expensive to fix & ensures a high-quality foundation before construction.
3. The use of scenarios & quality attributes.

4. To assess architectural decisions in light of quality attributes & business goals to discover risks & trade-offs.
5. End-user perspective, technical perspective, & business community perspective.
6. Evaluation team, project decision-makers, & architecture stakeholders.
7. A top-down approach to characterize & prioritize quality attribute requirements using scenarios.
8. A recap of the process, scenario analysis, explanation of candidate architectures/rationale, & a summary of work.
9. The 4+1 architectural view model.
10. It is faster (0.5 to 1 day), involves fewer people, & uses internal staff rather than outside experts.

Part 3: Essay Questions

Instructions: Use the provided source material to develop detailed responses to the following prompts.

1. **The Role of Stakeholders in ATAM:** Discuss the importance of involving a diverse group of stakeholders in the ATAM process. How does their participation in Step 7 (Brainstorming & Prioritizing Scenarios) help identify potential risks or misalignments with the architect's vision?
2. **The Dynamics of Quality Attribute Trade-offs:** Explain the concept that quality attributes can never be achieved in isolation. Using the ATAM framework, describe how architectural decisions act as a trade-off between competing goals like performance, security, & modifiability.
3. **The ATAM Team Structure:** Compare & contrast the responsibilities of the Team Leader, Evaluation Leader, & the two types of Scribes (Scenario & Proceedings). Why is it necessary to have such specialized roles during the evaluation?
4. **Phases of ATAM Execution:** Outline the four phases (0 through 3) of the ATAM process. Detail the typical duration, key participants, & primary activities for each phase, highlighting how the process moves from preparation to follow-up.

5. **Standard vs. Lightweight ATAM:** Analyze the circumstances under which an organization might choose a Lightweight ATAM over a standard evaluation. What are the potential drawbacks of using a faster, internal-only evaluation process?

Glossary of Key Terms

Term	Definition
ARID	Active Reviews for Intermediate Designs; an evaluation method using scenarios & quality attributes.
ATAM	Architecture Tradeoff Analysis Method; a specialized method to assess architectural decisions against quality attributes & business goals.
Business Drivers	The goals, context, constraints, & high-level requirements that motivate the creation of the software architecture.
Non-risk	An architectural decision that is deemed positive & supportive of the required quality attributes.
Quality Attribute	A property of a system (e.g., performance, security, or modifiability) used to judge its overall quality & effectiveness.
Quality Scenario	A specific description of a system's behavior used to characterize, prioritize, & evaluate quality attributes.
Risk	A potential problem identified during evaluation where an architectural decision may negatively affect a desired quality attribute.
SAAM	Software Architecture Analysis Method; an evaluation method sharing common scenario-based techniques with ATAM.
SAD	Software Architecture Document; the key artifact used to specify & document the selected system architecture.
Sensitivity Point	An architectural decision to which a particular quality attribute is highly sensitive.
Trade-off	A situation where achieving one quality attribute (e.g., high security) negatively impacts another (e.g., performance).
Utility Tree	A top-down hierarchical model used to identify & prioritize quality attribute goals & their associated scenarios.

Software Architecture Documentation

Executive Summary

Software architecture serves as the primary determinant of a system's success or failure. However, even the most robust architecture is ineffective if stakeholders cannot understand, apply, or modify it. Documentation is the essential bridge between architectural design & its practical implementation.

The core of architectural documentation lies in the selection & detail of specific “views,” supplemented by information that spans the entire system. Effective documentation serves three primary purposes: **Education** (onboarding project members), **Communication** (aligning stakeholders), & **Analysis** (validating quality attributes). To succeed, documentation must be stakeholder-centric, utilizing standard templates—such as those from the Rational Unified Process (RUP) or the Software Engineering Institute (SEI)—to ensure clarity & completeness while maintaining a “minimum necessary information” threshold.

The Strategic Importance of Documentation

The architecture of a software system is its most critical foundation. Without adequate documentation that conveys both functional requirements & quality attributes, a project is prone to failure.

- **Necessity of Communication:** Creating an architecture is only half the task; communicating it to stakeholders is equally vital.
- **Consequences of Poor Documentation:** Architecture becomes useless if those responsible for its use, construction, or modification:
 - Do not know the architecture exists.
 - Cannot understand it well enough to implement it.
 - Misinterpret the design & apply it incorrectly.

Primary Utility:

- **Education:** Introducing new personnel to the project.
- **Communication:** Facilitating dialogue & agreement among stakeholders.
- **Analysis:** Providing a basis for assuring quality attributes & validating design decisions.

Frameworks for Architectural Views

Documentation is structured around “views,” which represent specific aspects of the system. Several methodologies exist for selecting these views, with the **4+1 View Model** being the most commonly utilized.

Common Architectural View Approaches

Approach	Views Included	Primary Focus
Kruchten's 4+1	Logical, Process, Development, Physical + Use-Case	Behavioral requirements, concurrency, & mapping to physical nodes.
Siemens Four-Views	Conceptual, Module interconnection, Execution, Code	Structural & execution flow.
Herzum & Sims	Technical, Application, Project management, Functional	Management & technical integration.
Software Cost Reduction (SCR)	Module, Process, Uses	Encapsulation, performance, & incremental development.

The 4+1 View Model Detail

- **Logical View:** Focuses on behavioral requirements & key abstractions (objects/classes).
- **Process View:** Addresses concurrency, distribution, & the mapping of threads to objects.
- **Development View:** Outlines the organization of software modules, libraries, & units.
- **Physical View:** Maps software elements onto processing & communication nodes.
- **Use-Case View (The "+1"):** Relates all other views to important use cases to demonstrate system functionality.

The Documentation Process

A systematic approach to documentation involves four foundational steps:

1. **Define Stakeholders:** Identify all parties, including project managers, developers, testers, maintainers, customers, & end-users.
2. **Identify Concerns:** Record stakeholder concerns to prioritize which views are most relevant.
3. **Select Views:** Choose a set of views that is both minimal & adequate for the identified needs.
4. **Select Templates:** Use established standards (e.g., RUP or SEI) to define what & how to document.

Rational Unified Process (RUP) Template Structure

Based on Source Context the RUP template follows this hierarchy:

- **Introduction:** Purpose, Scope, & Overview.
- **Architectural Representation, Goals, & Constraints.**
- **Specific Views:** Use-Case, Logical (Design Packages), Process, Deployment, & Implementation (Layers).
- **Optional/Supplementary:** Data View, Size & Performance, & Quality.

Detailed Components of a Single View

Documenting a specific view requires more than a simple diagram. A comprehensive “view packet” includes:

1. **Primary Presentation:** Usually a graphical “cartoon” or a table showing elements & their relationships. It must include a key for notation.
2. **Element Catalog:** A detailed list explaining the elements in the primary presentation, their properties, interfaces, & relationship exceptions.
3. **Context Diagram:** A visual representation of how the system (or the specific portion of the view) relates to its external environment.
4. **Variability Guide:** Instructions on how to exercise variation points where decisions are left unbound until later development stages.
5. **Architecture Background:** The rationale for design decisions, including rejected alternatives, analysis results, & environmental assumptions.

Documentation Beyond the View

To provide a holistic understanding of the system, documentation must include information that applies to the entire architecture or links multiple views together.

- **Documentation Roadmap:** Guides stakeholders on how the documentation is organized & which scenarios require consulting specific parts.
- **System Overview:** A prose description of the system’s purpose, users, & constraints to create a consistent mental model.
- **Mapping Between Views:** Tables or descriptions that capture the associations & correspondences between different views.
- **Rationale for System-Wide Decisions:** Documentation of organizational constraints, major requirements, & the selection of fundamental architectural patterns.
- **Directory:** Reference materials including an index of terms, glossary, & acronym list.

Documenting Quality Attributes

Architecture documentation must explicitly address quality attribute requirements (e.g., performance, modifiability) & the tradeoffs made to achieve them.

- **Rationale & Tradeoffs:** The document must explain the choice of design approach in the context of quality requirements.
- **Service Bounds:** Architectural elements providing services should have assigned quality attribute bounds.

- **Requirement Mapping:** A mapping that demonstrates how specific requirements—particularly quality attributes—are satisfied by the architecture.
- **Stakeholder Assurance:** Since every quality attribute has a constituency of interested stakeholders, these attributes must be clearly identified & validated through analysis results.

Study Guide

This study guide provides a comprehensive review of the principles & practices involved in documenting software architectures. Based on the provided lecture outlines, it covers the necessity of documentation, various architectural view approaches, & the specific components required for thorough architectural records.

Part 1: Short-Answer Quiz

Instructions: Answer the following questions in two to three sentences based on the source context.

1. Why is documentation considered a critical determinant of a software system's success?
2. What are the three primary uses of architecture documentation?
3. What are the four initial steps required to start the documentation process?
4. In Kruchten's 4+1 views approach, what is the specific purpose of the Physical view?
5. How does the Software Cost Reduction (SCR) method utilize the "Module view"?
6. Why is it important to identify stakeholders & their concerns before documenting?
7. What information should be included in a "Primary Presentation" of an architectural view?
8. What is the purpose of an "Architecture Background" within the documentation of a view?
9. What does a "Documentation Roadmap" provide to a stakeholder?
10. How should quality attributes be represented in architectural documentation?

Part 2: Quiz Answer Key

1. **Why is documentation considered a critical determinant of a software system's success?** Software system architecture is the primary determinant of success; without adequate documentation, required

functions & quality attributes may not be delivered. Even a high-quality architecture is useless if stakeholders cannot understand, build, or modify it correctly due to poor communication.

2. **What are the three primary uses of architecture documentation?** The three primary uses are education, communication, & analysis. Documentation introduces new people to the project, facilitates essential discussion among stakeholders, & provides the necessary data for assuring quality attributes.
3. **What are the four initial steps required to start the documentation process?** To begin, an architect must select common views of the Software Architecture Design (SAD) & define the relevant stakeholders. Subsequently, they must identify those stakeholders' specific concerns & define what & how to document by selecting appropriate templates.
4. **In Kruchten's 4+1 views approach, what is the specific purpose of the Physical view?** The physical view is used to map software elements onto processing & communication nodes. It essentially defines how the software components are distributed across the physical hardware of the system.
5. **How does the Software Cost Reduction (SCR) method utilize the "Module view"?** The module view shows modules as units of encapsulation within the system. This specific view is used to isolate changes & achieve higher levels of modifiability.
6. **Why is it important to identify stakeholders & their concerns before documenting?** Identifying stakeholders & their concerns allows for the creation of a table that helps prioritize selected views based on specific needs. This ensures that the documentation addresses the most critical requirements, as some stakeholders may carry more weight than others.
7. **What information should be included in a "Primary Presentation" of an architectural view?** A primary presentation, which is usually graphical (a "cartoon") but can be textual, must show the elements & their relationships. It is essential that it includes a key explaining the notation used so that the information is clearly understood.
8. **What is the purpose of an "Architecture Background" within the documentation of a view?** The architecture background provides the rationale for design decisions, including rejected alternatives & factors that constrained the design. It also includes analysis results that validate these decisions & any assumptions made about the system's environment.
9. **What does a "Documentation Roadmap" provide to a stakeholder?** The roadmap explains how the documentation is organized & lists the available views along with their specific purposes. It also provides scenarios for using the documentation, directing stakeholders to the specific parts they should consult.
10. **How should quality attributes be represented in architectural documentation?** Documentation should clearly identify quality attributes & include a mapping to requirements to show how they are satisfied. It must also include a rationale for design approaches & discussions regarding tradeoffs & quality attribute bounds assigned to architectural elements.

Part 3: Essay Questions

Instructions: Use the source context to develop detailed responses for the following prompts.

1. **The Role of Communication in Architecture:** Analyze the statement that "communicating an architecture to its stakeholders is as important a job as creating it." Discuss the consequences of failing to document an architecture properly, even if the design itself is sound.
2. **Comparative Analysis of View Approaches:** Compare & contrast Kruchten's 4+1 views with the Software Cost Reduction (SCR) method. Discuss how each approach prioritizes different system goals, such as concurrency, modifiability, & performance.

3. **Standardization through Templates:** Evaluate the importance of using standard templates (such as the RUP or SEI templates) in software documentation. How do these templates ensure that the “minimum information necessary” is captured without overwhelming the reader?

4. **Information Beyond Views:** Explain the necessity of “documentation beyond views.” Why is it insufficient to document only the individual views of an architecture, & what critical information is captured in the system overview, mapping between views, & directory?

5. **Design Rationale & Decision Making:** Discuss the importance of documenting rationale & “rejected alternatives” in both the architecture background & the general documentation. How does this practice assist future maintainers & architects in understanding the system’s evolution?

Glossary of Key Terms

Term	Definition
Architecture Background	The rationale for design decisions, including rejected alternatives, analysis results, & environmental assumptions.
Context Diagram	A diagram showing how the system, or a portion of it, relates to its external environment.
Documentation Roadmap	A guide describing the organization of documentation, including a list of views & usage scenarios for stakeholders.
Element Catalog	A detailed list explaining the elements depicted in a primary presentation, including their properties, interfaces, & relations.
Kruchten's 4+1 Views	A popular architectural model consisting of Logical, Process, Development, & Physical views, plus a Use Case view to tie them together.
Mapping between Views	Documentation, often captured in tables, that establishes the correspondences & associations between different architectural views.
Primary Presentation	The initial graphical or textual display of a view showing elements & their relationships; must include a key for notation.
Rationale	Documentation of architectural decisions that apply to multiple views, often including organizational constraints or major requirements.
Software Architecture	A crucial determinant of system success that defines function & quality attributes.
Stakeholders	Individuals or groups with an interest in the architecture, including project managers, developers, testers, maintainers, & end users.
Variability Guide	A guide explaining how to exercise variation points in an architecture where decisions are left unbound until later development stages.
View	A representation of a set of system elements & the relationships between them, chosen to address specific stakeholder concerns.